NSCT Preparation
Programming (10%)

This file covers the core technical aspects of programming. It is language-agnostic but uses C/C++, Java, and Python-like syntax for examples. Each question shows the correct answer in **bold**, followed by the explanation immediately under it.

Master Table of Contents (File 02 - Target: 240 MCQs)

#	Subtopic	Coverage	MCQs
01	Programming Fundamentals	Standard	15
02	Data Types & Variables	Standard	15
03	Operators & Expressions	Standard	15
04	Control Structures	Extended	20
05	Functions / Methods	Extended	20
06	Input / Output Handling	Basic	10
07	Strings & Text Processing	Standard	15
08	Arrays & Collections	Extended	20
09	Object-Oriented Programming (OOP)	Extended	25
10	Memory Management Concepts	Basic	10
11	Exception & Error Handling	Standard	15
12	Modules, Packages & Libraries	Basic	10
13	Advanced Programming Concepts	Standard	15
14	Concurrency & Parallelism	Basic	10
15	Debugging, Testing & Optimization	Standard	15

16	Software Development Practices	Basic	10
Total MCQs			**240**

Subtopic 01: Programming Fundamentals (15 MCQs)

Quick Notes

- **Compiled vs. Interpreted:** Compiled languages (C/C++) translate code to machine code before execution (faster). Interpreted languages (Python/JS) translate code line-by-line during execution (more flexible).
- **Static vs. Dynamic Typing:** Static (Java/C++) checks types at compile-time. Dynamic (Python/JS) checks types at runtime.
- **Strong vs. Weak Typing:** Strong typing (Python/Java) prevents mixing incompatible types (e.g., adding string and int). Weak typing (C/JS) may allow implicit conversion (Type Coercion).
- **Paradigms:** 1) **Procedural:** Logic organized around functions/steps. 2) **Object-Oriented:** Logic organized around objects/data. 3) **Functional:** Logic organized around pure mathematical functions.
- **Compilation Process:** Pre-processing → Compilation (Lexical → Syntax → Semantic Analysis) → Assembly → Linking.

Common Traps & Confusions

- **Interpreted != Slow:** While generally slower, modern Just-In-Time (JIT) compilation makes interpreted languages like JavaScript/V8 very fast.
- **Type Safety vs. Static Typing:** A language can be Dynamically typed but still be Strongly typed (like Python), meaning it won't let you perform illegal operations between types.
- **The Linker's Role:** Syntax errors happen in the compiler. "Undefined reference" errors happen in the *Linker* (when a function definition is missing).

MCQs

Q1. (Conceptual) In the software build process, which stage is responsible for combining multiple object files and libraries into a single executable file?

A) Lexical Analyzer

B) Pre-processor

C) Assembler

D) Linker

> The Linker is the final stage. It resolves references between different files and combines them. If you call a function that is declared but not defined anywhere, the *Linker* will throw an error.

Q2. (Comparison) A language that requires you to declare `int x = 5;` and throws an error if you try to assign `x = "Hello";` later is best described as:

A) Dynamically Typed

B) Statically Typed

C) Weakly Typed

D) Interpreted

> Static typing means the variable's type is fixed at compile-time and cannot change. Java, C++, and Swift are statically typed languages.

Q3. (Scenario-Based) A developer is writing a high-performance graphics engine where every millisecond of CPU time matters. Which type of language should they prioritize?

A) A Compiled language like C++ or Rust

B) An Interpreted language like Python

C) A scripting language like Ruby

D) A markup language like HTML

> Compiled languages are translated directly into machine-specific binary before running. This avoids the overhead of line-by-line interpretation, offering the highest possible execution speed.

Q4. (Code-Tracing / Logic) In a **Weakly Typed** language like JavaScript, what is the likely result of the expression: `"5" + 2`?

A) 7

B) "52" (String concatenation)

C) Error: Type Mismatch

D) 5

> In weakly typed languages, the engine performs "Type Coercion." Since one operand is a string, it implicitly converts the integer 2 to a string and joins them. In a **Strongly Typed** language like Python, this would cause a crash.

Q5. (Conceptual) Which programming paradigm is built around the concept of "Pure Functions" that do not have side effects or shared state?

A) Object-Oriented Programming

B) Procedural Programming

C) Functional Programming

D) Low-level Assembly

> Functional programming (like Haskell or parts of Scala/JS) treats computation as the evaluation of mathematical functions, avoiding mutable data and changing state.

Q6. (Debugging) During compilation, the compiler finds a line of code: `if (x == 5) { y = 10 }` but notices the closing `)` is missing. Which phase of the compiler detects this?

A) Lexical Analysis

B) Syntax Analysis (Parsing)

C) Code Optimization

D) Code Generation

> Syntax analysis (Parsing) checks the structure of the code against the grammar rules of the language. A missing parenthesis is a grammatical (syntax) error.

Q7. (Comparison) What is the primary advantage of a **Managed Language** (like Java or C# with a Virtual Machine/CLR) over an **Unmanaged Language** (like C++)?

A) Managed languages run directly on the hardware.

B) Managed languages provide automatic memory management (Garbage Collection), reducing the risk of memory leaks.

C) Managed languages are always faster.

D) Managed languages allow direct pointer arithmetic.

> In managed environments, the runtime handles memory allocation and cleanup (Garbage Collection). In unmanaged languages, the programmer must manually free memory, which often leads to bugs if forgotten.

Q8. (Scenario-Based) You are creating a small script to automate a file-renaming task that will only be run once. You want to write it quickly without a "Build/Compile" step. Which type of language is best?

A) Statically Typed Compiled Language

B) Dynamically Typed Interpreted Language

C) Assembly Language

D) Parallel Language

> Interpreted languages (like Python or Bash) allow for "Read-Eval-Print-Loop" (REPL) development. You write the code and run it immediately, making them ideal for quick scripts and automation.

Q9. (Conceptual) What is the role of a "Virtual Machine" (like the JVM) in the execution of a program?

A) To speed up the internet connection.

B) To provide a platform-independent environment that executes intermediate "Bytecode".

C) To convert code into a PDF document.

D) To manage the hardware's cooling system.

> Languages like Java compile to "Bytecode," which is not machine-specific. The JVM then runs this bytecode on any OS (Windows, Mac, Linux), fulfilling the "Write Once, Run Anywhere" promise.

Q10. (Code Output / Logic) Trace the logic in this pseudocode:

```
x = "100"
y = 50
print(x + y)
```

If the language is **Strongly Typed** (like Python), what happens?

A) 150

B) "10050"

C) TypeError: cannot concatenate 'str' and 'int'

D) 100

> Strong typing prevents the implicit conversion of types. You cannot add a number to a string without explicitly converting one of them first (e.g., `int(x) + y`).

Q11. (Conceptual) In the context of programming paradigms, "Encapsulation" is a core principle of which paradigm?

A) Functional

B) Imperative

C) Object-Oriented

D) Logic-based

> Encapsulation involves bundling data and the methods that operate on that data into a single unit (Object), and restricting direct access to some of the object's components.

Q12. (Scenario-Based) A developer is writing code for a microcontroller with very limited RAM (only 2KB). Which language feature should they **avoid** to prevent crashing?

A) Conditional statements

B) Heavy use of Recursion (which fills the stack)

C) Global constants

D) Integer variables

> Recursion uses the "Call Stack" for every function call. On a memory-constrained device (microcontroller), a deep recursion will cause a "Stack Overflow" almost immediately.

Q13. (Comparison) What is a **Transpiler** (Source-to-Source Compiler)?

A) A tool that converts binary to assembly.

B) A tool that translates source code from one high-level language to another (e.g., TypeScript to JavaScript).

C) A tool that finds bugs in code.

D) A tool that compresses code for the web.

> A transpiler takes code in one language (like TypeScript) and produces equivalent code in another (like JavaScript) so that it can be run in environments (like browsers) that don't support the original language.

Q14. (Conceptual) What is "Semantic Analysis" in the compilation process?

A) Checking if the parentheses match.

B) Checking if the code makes logical sense (e.g., ensuring a variable is declared before use).

C) Converting code into binary zeros and ones.

D) Removing white space from the file.

> While Syntax analysis checks the structure, Semantic analysis checks the *meaning*. For example, `int x = "hello";` is syntactically correct (it follows the structure `type name = value;`), but semantically wrong because you can't put a string in an integer variable.

Q15. (Scenario-Based) You are using a language that uses a **Just-In-Time (JIT)** compiler. When does the actual machine code generation happen?

A) When you save the file.

B) Months before the program is run.

C) During the execution of the program (at runtime).

D) Only when the program crashes.

> JIT compilers (used in Java, C#, and JavaScript) compile frequently used "hot" sections of code into machine code while the program is running, combining the speed of compiled languages with the flexibility of interpreted ones.

Deep Dive

Programming Fundamentals provide the "Rules of the Game" for software development. Understanding these internals helps you choose the right tool for the job.

1. The Typing Matrix

Category	Static (Java/C++)	Dynamic (Python/Ruby)

Strong	Type-safe, checked at compile time.	Type-safe, checked at runtime.
Weak	Allows bit-level manipulation (C).	Heavy type coercion (JS/PHP).

2. The Compiler Pipeline

1. **Lexical Analysis:** Breaking code into "Tokens" (keywords, operators).
2. **Syntax Analysis:** Building a "Parse Tree" to check grammar.
3. **Semantic Analysis:** Checking types and declarations.
4. **Intermediate Code Gen:** Creating a generic "Bytecode" or "IR".
5. **Optimization:** Removing unused code and making it faster.
6. **Code Generation:** Producing the final machine code (Binary).

3. Multi-Paradigm Languages

Most modern languages (Python, Java, C++, JavaScript) are **Multi-paradigm**. This means you can write them using Procedural style (functions), Object-Oriented style (classes), or Functional style (lambda/map/filter), depending on what fits your specific problem best.

Stable Web Resources

- Wikipedia: Programming Paradigms
- GeeksforGeeks: Introduction to Compiler Design
- Wikipedia: Strong and Weak Typing

Quick Revision Cheat Sheet

- **Compiled:** Fast, ahead-of-time (C++, Rust).
- **Interpreted:** Flexible, line-by-line (Python, Ruby).
- **Static Type:** Type is checked by the compiler (Java).
- **Dynamic Type:** Type is checked by the runner (Python).
- **Linker:** Joins files together; throws "Undefined Reference" errors.
- **Functional:** No side effects, pure math.
- **JIT:** Hybrid speed (Java, JS).

END OF SUBTOPIC 01 — MOVING TO SUBTOPIC 02

Subtopic 02: Data Types & Variables (15 MCQs)

Quick Notes

- **Primitive Types:** Basic types provided by the language (int, char, float, double, boolean). They store actual values.
- **Reference Types:** Objects, Arrays, and Classes. They store *memory addresses* (pointers) to where the data is located.
- **Static vs. Final/Const:** `Static` variables belong to the class (shared by all objects). `Final/Const` variables cannot be changed once assigned.
- **Type Casting:** 1) **Implicit (Widening):** Small type to large type (e.g., int to double). Done automatically. 2) **Explicit (Narrowing):** Large type to small type (e.g., double to int). Can lead to data loss (truncation).
- **Size and Range:** A `byte` is 8 bits (-128 to 127). A 32-bit `int` ranges roughly ±2 billion. Exceeding this causes overflow.

Common Traps & Confusions

- **Floating Point Imprecision:** `0.1 + 0.2` is not `0.3` in binary. Never use floats for financial calculations; use `BigDecimal` or store cents as `int`.
- **Initialization:** Local variables in C++/Java are **not** automatically initialized. Using them before assignment leads to "Garbage Values" or compiler errors.
- **Character vs. String:** A `char` is a single 16-bit Unicode unit (in Java/C#). A `String` is an immutable object. Mixing them requires proper casting.

MCQs

Q1. (Conceptual) In memory management, where are **Primitive** variables usually stored compared to **Reference** objects?

A) Primitives on the Heap, Objects on the Stack.

B) Primitives (local) on the Stack, Objects on the Heap.

C) Both are always stored on the Heap.

D) Both are always stored on the Stack.

> Local primitive variables are stored on the Stack for fast access. Reference variables (the pointers) are on the Stack, but the actual Object they point to is stored on the Heap.

Q2. (Code-Tracing) What is the output of the following C++/Java code?

```
double myValue = 9.99;
int result = (int) myValue;
print(result);
```

A) 10 (Rounded up)

B) 9 (Truncated)

C) 9.99

D) Compilation Error

> Explicit casting from `double` to `int` performs **truncation**, not rounding. The decimal part is simply chopped off, resulting in 9.

Q3. (Scenario-Based) You are designing a system to store the population of the world (approx. 8 billion). Which data type is the most appropriate to prevent overflow?

A) 32-bit signed int

B) 32-bit unsigned int

C) 64-bit long (int64)

D) float

> A 32-bit signed int maxes at ~2.1 billion. A 32-bit unsigned int maxes at ~4.2 billion. To safely store 8 billion, a 64-bit long is required.

Q4. (Debugging) A developer writes: `float price = 19.99;` in Java/C#. The compiler throws an error: "Incompatible types: possible lossy conversion from double to float." Why?

A) Floats cannot hold decimals.

B) Floating-point literals are treated as 'double' by default; you must append 'f'

(19.99f).

C) 19.99 is too large for a float.

D) Price must be an integer.

> In Java/C#, decimal literals are `double` (64-bit). Assigning a 64-bit value to a 32-bit `float` requires explicit casting or the 'f' suffix to tell the compiler it's a float.

Q5. (Conceptual) What is the effect of the `static` keyword on a variable declared inside a class?

A) The variable cannot be changed after initialization.

B) The variable is shared across all instances (objects) of that class.

C) The variable is only accessible within the same package.

D) The variable is stored in the CPU cache for speed.

> A `static` variable belongs to the class itself, not any individual object. If one object changes a static variable, the change is reflected in all other objects of that class.

Q6. (Code-Tracing) Trace this logic:

```
int a = 127;
byte b = (byte) a;
b++;
print(b);
```

Assuming a standard 8-bit signed `byte` (-128 to 127):

A) 128

B) -128

C) 0

D) 1

> This is an **Overflow** trap. 127 is the maximum value for a byte (`01111111`). Adding 1 results in `10000000`, which in two's complement represents -128.

Q7. (Scenario-Based) An application needs to represent four specific states of an order: `PENDING`, `SHIPPED`, `DELIVERED`, `CANCELLED`. What is the most type-safe way to represent this?

A) Using four `String` constants.

B) Using integer values 0, 1, 2, 3.

C) Using an Enum (Enumerated Type).

D) Using a boolean array.

> Enums provide type safety. They prevent developers from passing invalid values (like 5 or "ON_WAY") to a function that expects only those four specific states.

Q8. (Comparison) What is the difference between `NULL` (C++) and `null` (Java/C#)?

A) Java's `null` is an integer 0.

B) C++ `NULL` is usually a macro for 0; Java/C# `null` is a literal representing an unassigned object reference.

C) There is no functional difference.

D) `null` can be used with primitives in Java.

> In C++, `NULL` is often just the number 0. In Java, `null` is a special value that can only be assigned to reference types (objects), not primitives.

Q9. (Conceptual) Why are `String` objects "Immutable" in languages like Java and Python?

A) To make them take up less memory.

B) For security, thread-safety, and allowing "String Pooling" to save memory.

C) Because characters cannot be changed in binary.

D) To prevent users from typing long sentences.

> Immutability means a string cannot be changed after creation. This allows the JVM/Interpreter to cache strings in a "Pool." If two variables hold "Hello," they can both point to the same memory location safely.

Q10. (Code-Tracing) What is the output of this code?

```
char c = 'A';
print(c + 1);
```

Assuming 'A' has an ASCII value of 65:

A) B

B) 66

C) A1

D) Error

> When you perform arithmetic on a `char`, it is implicitly promoted to an `int`. 65 + 1 = 66. To print 'B', you would need to cast it back: `(char)(c + 1)`.

Q11. (Scenario-Based) You are calculating the interest on a bank account with 10 million users. Using `float` for the balance results in a discrepancy of $50,000 across the whole system. What is the cause?

A) Integer Overflow

B) Accumulation of Floating-Point Rounding Errors

C) Division by Zero

D) Logic error in the loop

> Binary floating-point types (float/double) cannot represent many decimal fractions (like 0.1) exactly. Over millions of transactions, these tiny errors add up to significant financial discrepancies.

Q12. (Conceptual) What is a "Wrapper Class" in Java (e.g., `Integer` for `int`)?

A) A class used to encrypt integers.

B) A class that wraps a primitive type in an object so it can be used in Collections like ArrayList.

C) A faster version of the primitive type.

D) A class that prevents variables from being changed.

> Collections (like `List`) can only store objects, not primitives. Wrapper classes allow primitives to behave like objects when necessary.

Q13. (Comparison) What is the difference between `const` and `readonly` in C#?

A) `const` is a compile-time constant; `readonly` can be assigned a value at runtime (inside a constructor).

B) `const` is only for numbers.

C) `readonly` makes the code run faster.

D) They are identical.

> `const` values must be known when the code is written. `readonly` allows you to set a value once based on a calculation or input when the object is created.

Q14. (Code-Tracing) What is the value of `result`?

```
int x = 10;
float y = 4.0f;
auto result = x / y;
```

(Using C++/Java logic where `auto` or `var` infers the type)

A) 2

B) 2.0

C) 2.5

D) 2.500000

> When performing arithmetic between an `int` and a `float`, the `int` is promoted to a `float`. 10.0 / 4.0 = 2.5.

Q15. (Conceptual) Which of the following is an example of an "Unsigned" data type violation?

A) Storing 500 in a byte.

B) Assigning -5 to an unsigned integer.

C) Adding two large integers.

D) Dividing an integer by 2.

> An `unsigned` type cannot represent negative numbers. Assigning -5 will result in a very large positive number due to the way binary bits are interpreted (Two's complement wrap).

Deep Dive: Memory & Representation

Variables are more than just names; they are abstractions over memory addresses. A graduate developer must understand how they occupy physical space.

1. Stack vs. Heap

- **Stack:** Used for static memory allocation and local variables. It is very fast but has a limited size.
- **Heap:** Used for dynamic memory allocation (Objects/Arrays). It is larger but slower and requires manual or garbage-collected cleanup.

2. The Integer Overflow Trap

If you add 1 to the maximum value of a signed 32-bit integer (2,147,483,647), the binary representation "flips" the sign bit, and the number becomes -2,147,483,648. This is a common security vulnerability in C and C++.

Stable Web Resources

- Oracle: Java Data Types Official Guide
- Wikipedia: Floating-Point Arithmetic

Quick Revision Cheat Sheet

- **Primitives:** Store actual data (int, char). **References:** Store addresses (Objects).
- **Casting:** Widening (int to double) is safe; Narrowing (double to int) loses data.
- **Immutable:** Strings cannot be changed (in Java/Python). This allows for string pooling.
- **Static:** Variable is shared by all objects of the class.
- **Final/Const:** Value cannot be changed once set.

END OF SUBTOPIC 02 — MOVING TO SUBTOPIC 03

Subtopic 03: Operators & Expressions (15 MCQs)

Quick Notes

- **Arithmetic Operators:** `+`, `-`, `*`, `/`, `%` (Modulo/Remainder).
- **Increment/Decrement:** 1) **Prefix (++x):** Increment first, then use. 2) **Postfix (x++):** Use first, then increment.
- **Logical Operators:** && (AND), `||` (OR), `!` (NOT). *Features Short-Circuit Evaluation.*
- **Bitwise Operators:** & (AND), `|` (OR), ^ (XOR), ~ (NOT), << (Left Shift), >> (Right Shift).
- **Ternary Operator:** `condition ? value_if_true : value_if_false`. A shortcut for if-else.
- **Operator Precedence:** Multiplication/Division/Modulo > Addition/Subtraction > Logical.

Common Traps & Confusions

- **Integer Division:** `5 / 2` is 2, not `2.5`. The decimal is truncated.
- **Short-Circuiting Side Effects:** If you have `if (A && B++)` and A is False, `B++` **never runs**. This can lead to hidden bugs where variables don't update as expected.
- **Assignment vs. Equality:** `if (x = 5)` vs `if (x == 5)`. In C/C++, the first one sets x to 5 and is always True!

MCQs

Q1. (Code-Tracing) What is the final value of `z`?

```
int x = 5;
int y = 2;
int z = x / y * y;
```

A) 5

B) 4

C) 5.0

D) 2.5

> Operators are left-associative.
> Step 1: `5 / 2` (Integer division) = 2.
> Step 2: `2 * 2` = 4.

Q2. (Code-Tracing) Trace the value of `x` and `y`:

```
int x = 10;
int y = x++;
```

A) x=11, y=11

B) x=11, y=10

C) x=10, y=11

D) x=10, y=10

> This is **Postfix** increment. `y` is assigned the current value of `x` (10), and *then* `x` is incremented to 11.

Q3. (Scenario-Based) Which operator is best used to check if a specific number `N` is a multiple of 10?

A) `N / 10 == 0`

B) `N % 10 == 0`

C) `N & 10 == 0`

D) `N || 10`

> The modulo operator (`%`) returns the remainder. If a number is a multiple of 10, the remainder when divided by 10 will be exactly 0.

Q4. (Debugging) Trace the following logic:

```
int count = 0;
if (true || (++count > 0)) {
    print(count);
}
```

A) 1

B) 0

C) true

D) Error

> This is **Short-Circuit Evaluation**. Because the first part of the `OR (||)` is `true`, the second part `(++count > 0)` is never even looked at by the computer. Therefore, `count` remains 0.

Q5. (Calculation) What is the result of the bitwise **XOR (^)** operation on 5 and 3?

A) 8

B) 2

C) 6

D) 7

> 5 in binary: `101`
> 3 in binary: `011`
> XOR result: `110` (which is 6 in decimal).
> XOR is 1 if bits are different, 0 if they are same.

Q6. (Code-Tracing) What will be printed?

```
int x = 10;
int y = 20;
print(x > 5 && y < 15 ? "Alpha" : "Beta");
```

A) Alpha

B) Beta

C) True

D) 10

> Condition: `(10 > 5)` is True AND `(20 < 15)` is False.
> `True && False` results in False. The ternary operator returns the second option: "Beta".

Q7. (Conceptual) What does performing a **Bitwise Left Shift (<<)** by 2 positions do to an integer?

A) Divides it by 4.

B) Multiplies it by 2.

C) Multiplies it by 4.

D) Adds 2 to it.

> Each left shift by 1 position effectively multiplies the number by 2. Shifting by 2 positions multiplies it by 2^2 = 4.

Q8. (Code-Tracing) Trace the value of `val`:

```
int val = 8;
val >>= 2;
```

A) 16

B) 4

C) 2

D) 1

> `>>=` 2 is Right Shift by 2. 8 in binary is `1000`.
> Shift right once: `0100` (4).
> Shift right twice: `0010` (2).

Q9. (Comparison) What is the difference between & and && in Java/C#?

A) There is no difference.

B) & is for booleans, && is for integers.

C) && is a logical operator with short-circuiting; & is a bitwise operator that always evaluates both sides.

D) & is faster than &&.

> If you use & with booleans, it will check the second condition even if the first is already false. && will stop immediately if the first is false.

Q10. (Calculation) Evaluate the expression: `10 + 5 * 2 == 20`

A) True

B) False

C) 30

D) Error

> Precedence: `*` is higher than `+`.
> Step 1: `5 * 2 = 10`.
> Step 2: `10 + 10 = 20`.
> Step 3: `20 == 20` is True.

Q11. (Scenario-Based) A developer needs to swap two integers `a` and `b` without using a third temporary variable. Which bitwise operator can achieve this?

A) AND (&)

B) OR (|)

C) XOR (^)

D) NOT (~)

> The "XOR Swap Algorithm" uses `a = a ^ b; b = a ^ b; a = a ^ b;` to swap values without extra memory.

Q12. (Debugging) Trace this code snippet:

```
int x = 5;
if (x = 0) {
    print("Zero");
} else {
    print("Non-Zero");
}
```

In **C/C++**, what is the output?

A) Zero

B) Non-Zero

C) Error

D) 5

> In C++, `x = 0` is an **assignment**, not a comparison. It sets x to 0 and returns the value 0. In C++, 0 is treated as `false`. Therefore, it executes the `else` block.

Q13. (Code-Tracing) Trace the output:

```
int i = 5;
int j = ++i + i++;
print(j);
```

A) 10

B) 11

C) 12

D) 13

> Step 1: `++i` increments `i` to 6 and returns 6.
> Step 2: `i++` uses the current value of `i` (which is now 6) and *then* increments it to 7.
> Step 3: `6 + 6 = 12`.

Q14. (Conceptual) Which operator has the lowest precedence in most programming languages?

A) + (Addition)

B) && (Logical AND)

C) == (Equality)

D) = (Assignment)

> Assignment is almost always the last thing to happen in an expression so that the entire right-hand side can be calculated first.

Q15. (Code-Tracing) Trace the value of `ans`:

```
int ans = 10 % 3 + 4 << 1;
```

A) 10

B) 4

C) 2

D) 12

> Precedence: `%` > + > <<.
> Step 1: `10 % 3 = 1`.
> Step 2: `1 + 4 = 5`. (Wait, check precedence: + is higher than <<).
> Step 3: `5 << 1 = 10`.

> **Wait, Correction:** Modulo (1) + 4 = 5. Left shift 1 = 10.
> *Correction on common implementation:* In C++/Java, + is higher than <<. `10 % 3` (1) `+ 4` = 5. `5 << 1 = 10`.
> (Note: Always check language-specific tables, but + is usually higher than bit-shifts).

Deep Dive: Precedence & Logic

1. Short-Circuiting for Safety

The most common professional use of short-circuit evaluation is **Null Checking**. Example: `if (user != null && user.age > 18)`. If the first part is false, the second part (which would crash the program) is never executed.

2. Bitwise vs. Arithmetic

Bitwise operations (<<, >>) are significantly faster than arithmetic operations (`*`, `/`) at the hardware level. Compilers often replace `x * 2` with `x << 1` during optimization.

Stable Web Resources

- CppReference: Operator Precedence Table
- GeeksforGeeks: Advanced Bitwise Operations

Quick Revision Cheat Sheet

- **PEMDAS/BODMAS:** Operators follow strict math hierarchy. Parentheses always come first.
- **Prefix vs Postfix:** ++x (Change then use); x++ (Use then change).
- **Assignment:** = is the lowest precedence operator.
- **Bitwise XOR (^):** Returns 1 only if bits are different. Used for encryption and swapping.
- **Modulo (%):** Returns the remainder. Essential for circular logic and parity checks.

END OF SUBTOPIC 03 — MOVING TO SUBTOPIC 04

Subtopic 04: Control Structures (20 MCQs)

Quick Notes

- **Selection:** `if-else` is used for range-based or complex logic. `switch-case` is used for discrete, constant values and is often optimized by the compiler using a "Jump Table."
- **Iteration:** 1) **for:** Best for definite loops. 2) **while:** Best for indefinite loops (pre-test). 3) **do-while:** Guarantees at least one execution (post-test).
- **Jump Statements:** `break` terminates the nearest loop. `continue` skips to the next iteration. `return` exits the entire function.
- **Nesting:** Placing one control structure inside another. Complexity increases exponentially (O(N^2), O(N^3)).
- **Fall-through:** In a switch-case, execution continues to the next case unless a `break` is encountered.

Common Traps & Confusions

- **Dangling Else:** In many languages, an `else` always attaches to the *nearest* preceding `if` unless braces `{}` are used.
- **Empty Loop Body:** A semicolon after a loop declaration (e.g., `for(...);`) creates an empty loop body, often causing logical errors where only the last value is processed.
- **Floating Point Loops:** Using `float` in a loop condition (e.g., `while(x != 1.0)`) is dangerous because `x` might never exactly equal `1.0` due to precision errors, causing an infinite loop.

MCQs

Q1. (Code-Tracing) What is the output of the following C++/Java code?

```
int x = 2;
switch (x) {
    case 1: print("A");
    case 2: print("B");
    case 3: print("C");
    default: print("D");
}
```

A) B

B) BCD

C) BCD (due to missing break statements)

D) Error

> Because there are no `break` statements, once the program matches `case 2`, it "falls through" and executes every subsequent case, including the `default`.

Q2. (Debugging) A developer wants a loop that runs exactly 10 times. Which of the following contains a **logical error**?

```
A) for (int i = 0; i < 10; i++)
B) for (int i = 1; i <= 10; i++)
C) int i = 0; while (i < 10) { i++; }
D) int i = 10; while (i > 0) { i--; }
```

A, B, and C are correct.

D) None of the above (all are correct).

> All four structures will iterate exactly 10 times. A: 0-9. B: 1-10. C: 0-9. D: 10-1. This is a common test of "Off-By-One" awareness.

Q3. (Code-Tracing) How many times will "Hello" be printed?

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        if (i == j) continue;
        print("Hello");
    }
}
```

A) 6

B) 4

C) 3

D) 2

> Outer i: 0, 1, 2. Inner j: 0, 1.
> (0,0): i==j → skip.
> (0,1): Hello (1).
> (1,0): Hello (2).
> (1,1): i==j → skip.
> (2,0): Hello (3).
> (2,1): Hello (4). Total = 4.

Q4. (Conceptual) Why is a `do-while` loop generally preferred over a `while` loop for displaying a user menu?

A) It runs faster on the CPU.

B) It uses less memory.

C) It ensures the menu is displayed at least once before checking if the user wants to exit.

D) It prevents infinite loops.

> In a menu system, you must show the options at least once. A `while` loop checks the condition *first*, which might prevent the menu from ever appearing if the exit condition is somehow met initially.

Q5. (Scenario-Based) An algorithm's performance is $O(N^2)$. This is most likely caused by which control structure?

A) A single `for` loop running N times.

B) Two nested `for` loops, both running N times.

C) A `switch` statement with N cases.

D) An `if-else` chain with N conditions.

> In nested loops, the total iterations are N * N. This quadratic growth ($O(N^2)$) is the "fingerprint" of many inefficient algorithms (like Bubble Sort).

Q6. (Code-Tracing) Trace the value of `sum`:

```
int sum = 0;
for (int i = 0; i < 5; i++) {
    if (i % 2 == 0) sum += i;
    else if (i == 3) break;
    else sum -= 1;
}
```

A) 6

B) 2

C) 1

D) 0

> i=0: sum = 0+0 = 0.
> i=1: sum = 0-1 = -1.
> i=2: sum = -1+2 = 1.
> i=3: i==3 → BREAK.
> Final sum = 1.

Q7. (Conceptual) What happens when a `break` statement is executed inside the **inner** loop of two nested loops?

A) Both loops terminate immediately.

B) Only the inner loop terminates; the outer loop continues to its next iteration.

C) The entire program terminates.

D) The inner loop skips the current iteration but continues the next one.

> A `break` only affects the *innermost* loop that contains it. To break out of both, you would need a "Labelled Break" (in Java/JS) or a boolean flag.

Q8. (Debugging) Look at this loop carefully. What is the outcome?

```
for (double i = 0.0; i != 1.0; i += 0.1) {
    print(i);
}
```

A) It prints 0.0 to 0.9 and stops.

B) It likely results in an infinite loop due to floating-point precision.

C) It prints 0.0 to 1.0 and stops.

D) It throws a compilation error.

> 0.1 cannot be represented exactly in binary. The sum `i += 0.1` will eventually be `0.999999999...` or `1.000000001...`, meaning `i != 1.0` will stay true forever.

Q9. (Code-Tracing) What is the final value of `x`?

```
int x = 0;
while (x < 10) {
    if (x < 5) x += 2;
    else x += 5;
}
```

A) 10

B) 11

C) 12

D) 15

> Start: x=0.
> Loop 1: x=2.
> Loop 2: x=4.
> Loop 3: x=6.
> Loop 4: (x is not < 5), x = 6 + 5 = 11.
> Loop 5: (11 < 10) is False. Exit. Final x=11.

Q10. (Conceptual) Which control structure is best suited for implementing a "State Machine" where a variable can be in one of many discrete states?

A) Nested `for` loops

B) `if-else` chain

C) `switch-case`

D) `do-while`

> State machines transition between named states (e.g., IDLE, RUNNING, ERROR). A `switch` statement is the most readable and efficient way to map these discrete integer or enum-based states.

Q11. (Code-Tracing) What will be printed?

```
int i = 0;
for (; i < 3; i++);
print(i);
```

A) 0 1 2

B) 2

C) 3

D) Nothing (Infinite loop)

> There is a semicolon `;` immediately after the `for` loop header. This means the loop body is empty. The loop runs 3 times (i becomes 0, 1, 2, then 3). When i=3, the loop ends, and 3 is printed.

Q12. (Comparison) What is the "Dangling Else" problem?

A) An `else` statement with no `if`.

B) An `if` statement that never executes.

C) Ambiguity in nested `if-else` statements regarding which `if` the `else` belongs to.

D) A memory leak caused by conditional branches.

> Without braces `{}`, it's unclear if an `else` belongs to the outer or inner `if`. Compilers solve this by attaching it to the *closest* `if`, but it can cause major logic bugs for the developer.

Q13. (Scenario-Based) In a video game, you want a "Gravity" function to run every frame unless the game is "Paused." Which structure is most logical?

A) `switch (paused)`

B) `if (!paused) { applyGravity(); }` inside the game loop.

C) A `while (!paused)` loop inside the gravity function.

D) `do { applyGravity(); } while (!paused);`

> Game logic typically uses a main game loop. You check the state (paused) using an `if` in each frame to decide whether to execute specific logic.

Q14. (Calculation) How many times will the condition in this `while` loop be evaluated?

```
int x = 0;
while (x < 5) {
    x++;
}
```

A) 4

B) 5

C) 6

D) 0

> The condition is checked 5 times where it is True (x=0,1,2,3,4). It is checked 1 final time where it is False (x=5) to terminate the loop. Total = 6 checks.

Q15. (Code-Tracing) Trace the output of this C++ snippet:

```
int n = 1;
label:
print(n);
n++;
if (n <= 3) goto label;
```

A) 1

B) 1 2 3

C) 1 2

D) Error

> A `goto` statement combined with a label can create a loop. This is functionally equivalent to a `do-while` loop, printing 1, 2, and 3. (Note: `goto` is discouraged in professional code but valid syntax).

Q16. (Conceptual) What is the "Short-Circuit" effect in the context of an `if` statement condition?

A) The program crashes to prevent damage.

B) The second half of a logical expression is skipped if the result is already determined by the first half.

C) All loops are skipped.

D) Variables are reset to zero.

> In `if (A || B)`, if A is true, B is not checked. In `if (A && B)`, if A is false, B is not checked. This is efficient and prevents errors (like null checks).

Q17. (Code-Tracing) Trace the output:

```
for (int i = 0; i < 10; i++) {
    if (i % 3 == 0) continue;
    if (i > 5) break;
    print(i);
}
```

A) 0 1 2 3 4 5

B) 1 2 4 5

C) 1 2 3 4 5

D) 0 1 2 4 5

> i=0: 0%3==0 → continue.
> i=1: print 1.
> i=2: print 2.
> i=3: 3%3==0 → continue.
> i=4: print 4.
> i=5: print 5.
> i=6: i>5 → BREAK. Final: 1 2 4 5.

Q18. (Debugging) Why is `while(true)` without a `break` inside usually a critical bug in a standard desktop application?

A) It makes the code file larger.

B) It causes "Application Hang" (UI Unresponsiveness) by blocking the main thread.

C) It deletes temporary files.

D) It is only allowed in Linux.

> The UI runs on a main thread. If that thread enters an infinite loop, it can no longer process user clicks or window repaints, making the app appear "frozen."

Q19. (Comparison) When is an `if-else if` chain more efficient than multiple `if` statements?

A) Always.

B) When the conditions are mutually exclusive (only one can be true).

C) When you need to check multiple independent flags.

D) When the variables are strings.

> With `else if`, once a condition is met, all subsequent conditions are skipped. With multiple `if` statements, the computer must check every single one, even if the first one was already true.

Q20. (Code-Tracing) What is the output?

```
int a = 10, b = 20;
int result = (a > b) ? (a + b) : (b - a);
print(result);
```

A) 30

B) 10

C) -10

D) 0

> The condition `(10 > 20)` is False. The ternary operator returns the value after the colon: `b - a`, which is `20 - 10 = 10`.

Deep Dive: Flow Optimization

1. Jump Tables in Switch-Case

Compilers often optimize `switch` statements using a **Jump Table**. Unlike an `if-else if` chain, which checks every condition one by one (O(N)), a jump table can jump directly to the correct case in constant time (O(1)).

2. Loop Unrolling

To improve performance, compilers perform "Loop Unrolling," where they repeat the loop body multiple times in the binary code to reduce the number of times the "loop condition" needs to be checked.

Stable Web Resources

- Wikipedia: Control Flow Structures
- W3Schools: Switch Case Logic

Quick Revision Cheat Sheet

- **Break:** Exits the current loop entirely.
- **Continue:** Skips current iteration and starts the next one.
- **Do-While:** The code body is guaranteed to run at least 1 time.
- **Fall-through:** Occurs in Switch if you forget the `break` keyword.
- **Dangling Else:** Solved by using curly braces `{}` to clarify logic.

END OF SUBTOPIC 04 — MOVING TO SUBTOPIC 05

Subtopic 05: Functions / Methods (20 MCQs)

Quick Notes

- **Definition:** A block of reusable code that performs a specific task. Components: Name, Return Type, Parameters, Body.
- **Pass-by-Value:** A *copy* of the data is passed. Changes inside the function do not affect the original variable.
- **Pass-by-Reference:** The *memory address* is passed. Changes inside the function **do** affect the original variable.
- **Scope:** Local variables exist only inside the function. Global variables exist for the life of the program.
- **Recursion:** A function calling itself. Must have a **Base Case** to stop and a **Recursive Step** to reach it.

- **Overloading:** Multiple functions with the same name but different parameters (compile-time polymorphism).

Common Traps & Confusions

- **Stack Overflow:** Occurs when a recursive function never hits its base case, filling the stack memory with function frames until it crashes.
- **Returning Local References:** In C++, returning a pointer to a local variable is a "Dangling Pointer" bug because that memory is freed when the function ends.
- **Java/Python Reference Trap:** In these languages, objects are passed by *value of the reference*. This means you can change the internal data of an object, but you cannot make the original variable point to a new object.

MCQs

Q1. (Conceptual) What happens to a function's local variables when the function finishes executing?

A) They are moved to the Heap for storage.

B) They remain in memory until the program ends.

C) Their stack frame is popped, and the memory is reclaimed.

D) They are automatically converted to global variables.

> Local variables live on the Stack. When a function returns, its "Frame" is removed (popped) from the stack, and that memory is immediately available for the next function call.

Q2. (Code-Tracing) Trace the value of n in this "Pass-by-Value" example:

```
void doubleVal(int x) {
    x = x * 2;
}
int n = 10;
doubleVal(n);
print(n);
```

A) 20

B) 10

C) 0

D) Error

> Because `n` is a primitive passed by value, the function receives a copy. The calculation `x = 20` only changes the copy inside the function. The original `n` remains 10.

Q3. (Scenario-Based) You are writing a recursive function to calculate a factorial. What is the most important component to prevent the program from crashing?

A) A large integer return type.

B) A for loop inside the function.

C) A Base Case (e.g., if n <= 1 return 1).

D) A global counter.

> Without a base case, the function will call itself infinitely, creating new stack frames until the program runs out of stack memory (Stack Overflow).

Q4. (Code-Tracing / Java Logic) What is the output of this snippet?

```
void changeList(List list) {
    list.add("B");
}
List myL = new ArrayList();
myL.add("A");
changeList(myL);
print(myL.size());
```

A) 1

B) 2

C) 0

D) Error

> In Java, objects are passed by value of the reference. The function receives a copy of the "remote control" to the list. Using that control to `.add()` modifies the actual object on the heap.

Q5. (Conceptual) Function **Overloading** is an example of which type of polymorphism?

A) Compile-time (Static) Polymorphism

B) Runtime (Dynamic) Polymorphism

C) Recursive Polymorphism

D) Inline Polymorphism

> Overloading is resolved by the compiler based on the function signature (name + parameters). It decides which version to call *before* the program ever runs.

Q6. (Debugging) Why will the following recursive function crash for `sum(3)`?

```
int sum(int n) {
    return n + sum(n - 1);
}
```

A) It uses too much CPU.

B) You cannot add `n` to a function.

C) It lacks a base case, causing infinite recursion.

D) It should be `sum(n + 1)`.

> It will call `sum(3) → sum(2) → sum(1) → sum(0) → sum(-1)` and so on forever until a Stack Overflow occurs.

Q7. (Scenario-Based) A developer wants to return **two** different values (a result and an error code) from a single function in C++. What is the best modern approach?

A) Use two `return` statements.

B) Use a global variable for the error code.

C) Return a `struct` or `pair` containing both values.

D) Print the error code instead of returning it.

> Functions can only return one "thing." To return multiple data points, you must bundle them into a single Object or Structure.

Q8. (Code-Tracing) Trace the value of `result`:

```
int calculate(int a, int b = 5) {
    return a + b;
```

```
}
int result = calculate(10);
```

A) 10

B) 15

C) Error: Missing parameter

D) 5

> This uses a **Default Parameter**. If the second argument is not provided, the function uses the default value of 5. `10 + 5 = 15`.

Q9. (Conceptual) What is a "Pure Function" in the context of functional programming?

A) A function that doesn't use any variables.

B) A function that always returns the same output for the same input and has no side effects.

C) A function that is written in assembly.

D) A function that only handles integer data.

> Pure functions do not change global variables, write to files, or modify their inputs. They are predictable and easy to test.

Q10. (Code-Tracing) What will be printed?

```
int x = 50;
void doSomething() {
    int x = 100;
}
doSomething();
print(x);
```

A) 100

B) 50

C) Error

D) Null

> The `x` inside the function is a **Local Variable**. It "shadows" the global `x`, but only within that function's scope. The global `x` remains unchanged at 50.

Q11. (Comparison) What is the difference between a "Parameter" and an "Argument"?

A) They are the same thing.

B) Parameters are variables in the function definition; Arguments are the actual values passed during the call.

C) Arguments are for math; Parameters are for strings.

D) Parameters are global; Arguments are local.

> Definition: `void fun(int p)` → `p` is a parameter.
> Call: `fun(10)` → `10` is the argument.

Q12. (Calculation) Trace this recursive function: `f(n) = f(n-1) + f(n-2)`. Given `f(0)=0, f(1)=1`, what is `f(4)`?

A) 2

B) 3

C) 4

D) 5

> f(2) = 1+0 = 1.
> f(3) = 1+1 = 2.
> f(4) = 2+1 = 3. (This is the Fibonacci sequence).

Q13. (Conceptual) What does the `void` keyword indicate in a function signature?

A) The function can take any type of input.

B) The function does not return any value to the caller.

C) The function is empty and has no code.

D) The function is private to the class.

> `void` means the function performs an action (like printing) but does not send data back via a `return` statement.

Q14. (Scenario-Based) A developer wants to pass a massive 1GB object to a function in C++. Which method should they use to avoid slowing down the program?

A) Pass by Value

B) Pass by Reference (or Pointer)

C) Pass as a String

D) Pass by Global Variable

> Pass-by-value would create a 1GB copy of the object in memory, which is slow and wasteful. Pass-by-reference only passes a 64-bit memory address, which is near-instant.

Q15. (Debugging) What is wrong with this C++ function?

```
int* getPointer() {
    int val = 10;
    return &val;
}
```

A) Nothing, it returns the address of 10.

B) It returns a reference to a local variable that will be destroyed when the function exits (Dangling Pointer).

C) `&val` should be `*val`.

D) You cannot return pointers from functions.

> `val` lives on the stack frame of `getPointer`. As soon as the function returns, that frame is destroyed. The returned pointer now points to "garbage" memory.

Q16. (Code-Tracing) What is the output of this Python-like snippet?

```
def multiply(a, b):
    return a * b

print(multiply(b=3, a=2))
```

A) 2

B) 3

C) 6

D) Error

> This uses **Keyword Arguments**. The order doesn't matter as long as the parameters are explicitly named. `2 * 3 = 6`.

Q17. (Conceptual) In modern languages, what is a "Lambda" function?

A) A function that only works with Greek characters.

B) An anonymous, short function that can be treated as a variable.

C) A function that runs on a separate server.

D) A function that cannot be overloaded.

> Lambdas (or Arrow functions) are used for quick, one-line logic, often passed as arguments to other functions like `map` or `filter`.

Q18. (Scenario-Based) You have a function `saveData()` that is called 1 million times. You notice that the "overhead" of calling the function is slowing down the program. Which optimization might help?

A) Make the function recursive.

B) Use "Inlining" (requesting the compiler to replace the call with the actual code).

C) Make the function static.

D) Add more parameters.

> Function calls involve jumping to a new memory address and setting up a stack frame. For tiny, frequent functions, "Inlining" removes this overhead by pasting the code directly at the call site.

Q19. (Code-Tracing) What is the output?

```
int x = 10;
void update() {
    x = 20;
}
update();
print(x);
```

A) 10

B) 20

C) Error

D) 0

> Since `x` is not declared inside the function, it refers to the **Global Variable**. Calling `update()` changes the global value to 20.

Q20. (Conceptual) What is the "Call Stack"?

A) A list of all variables in a database.

B) A LIFO data structure that stores information about the active functions of a computer program.

C) A backup of the program on the hard drive.

D) A way to sort functions alphabetically.

> When you call a function, it is "pushed" onto the stack. When it returns, it is "popped." This allows the computer to keep track of where to go back to when a function finishes.

Deep Dive: The Function Lifecycle

1. Activation Records

Every time a function is called, an **Activation Record** (Stack Frame) is created. It stores: 1) Return Address, 2) Parameters, 3) Local Variables. If you call too many functions (deep recursion), the Stack memory is exhausted.

2. Overloading vs. Overriding

- **Overloading:** Same name, different parameters in the **same class** (Compile-time).
- **Overriding:** Same name and parameters in a **child class** (Runtime).

Stable Web Resources

- Wikipedia: Recursion in Computer Science
- Oracle: Defining and Calling Methods

Quick Revision Cheat Sheet

- **Pass-by-Value:** Function gets a copy. Original variable is safe.
- **Pass-by-Reference:** Function gets the address. Original variable is modified.
- **Recursion:** Needs a Base Case to stop and a Recursive Step to reach it.
- **Void:** Return type used when a function returns no data.
- **Shadowing:** When a local variable has the same name as a global variable.

END OF SUBTOPIC 05 — MOVING TO SUBTOPIC 06

Subtopic 06: Input / Output Handling (10 MCQs)

Quick Notes

- **Standard Streams:** 1) `stdin` (Input - usually keyboard), 2) `stdout` (Output - usually console), 3) `stderr` (Error - for logging errors separately from data).
- **Buffering:** Data is often collected in a "buffer" (temporary RAM) and written to the disk in chunks to improve performance. Using `flush()` forces the buffer to empty.
- **Text vs. Binary I/O:** Text mode handles character encoding (like newlines `\n`). Binary mode handles raw bytes (images, compiled code).
- **File Access:** Files must be `opened`, `processed`, and critically, `closed` to release system resources.
- **Serialization:** The process of converting an object in memory into a byte stream so it can be saved to a file or sent over a network.

Common Traps & Confusions

- **Resource Leaks:** Forgetting to close a file or network socket. In modern languages, the `try-with-resources` or `using` block handles this automatically.
- **The `stderr` Advantage:** If you redirect output to a file (e.g., `myapp > out.txt`), messages sent to `stdout` go to the file, but `stderr` still prints to the screen.
- **Scanner vs. BufferedReader:** In Java, `Scanner` is easier for parsing tokens, but `BufferedReader` is significantly faster for reading large files.

MCQs

Q1. (Conceptual) When a program needs to report a critical error that should not be mixed with the regular program output, which standard stream should it use?

A) stdin

B) stdout

C) stderr

D) stdlog

> `stderr` is the standard error stream. It is unbuffered in many environments and is specifically designed to keep error messages separate from the standard output (stdout), which might be piped to another process or file.

Q2. (Code-Tracing) Trace the file handling logic in this pseudocode:

```
file = open("data.txt", "w")
file.write("Hello")
# System crashes here
file.close()
```

What is a likely real-world risk if the `close()` method is never reached?

A) The file will be deleted automatically.

B) The data "Hello" may not actually be saved to the disk due to buffering.

C) The hard drive will experience physical damage.

D) The text "Hello" will be converted to binary.

> I/O is usually buffered in RAM. Data is only physically written to the disk when the buffer is full or the file is closed. A crash before `close()` or `flush()` often results in data loss.

Q3. (Scenario-Based) A developer opens a log file using the mode `"w"` (write). The file already contains 100 lines of logs. What happens to those 100 lines after the file is opened?

A) The new logs are added to the end.

B) The existing 100 lines are deleted (overwritten).

C) The computer throws an error because the file isn't empty.

D) The file becomes read-only.

> In standard I/O (C, Python, Java), the `"w"` mode truncates the file to zero length if it exists. To keep existing data, the `"a"` (append) mode must be used.

Q4. (Conceptual) What is the main difference between reading a file in "Text" mode versus "Binary" mode?

A) Text mode is faster.

B) Binary mode can only be used for images.

C) Text mode may automatically translate end-of-line characters (like \r\n to \n); Binary mode reads bytes exactly as they are.

D) Text mode does not require the file to be closed.

> Text mode is aware of character encodings and OS-specific line endings. Binary mode is "raw"—it returns the exact bits from the drive without any interpretation.

Q5. (Debugging) Look at this C-style code: `printf(user_input);`. Why is this considered a high-severity security vulnerability?

A) It uses too much memory.

B) It is a "Format String Attack" vulnerability; if the user inputs "%s" or "%n", they can read/write to the program's memory.

C) It is slow because it prints to the console.

D) It only works with strings, not integers.

> The first argument of `printf` should be a fixed format string (e.g., `printf("%s", user_input);`). Passing user input directly allows a malicious user to inject format specifiers and manipulate the stack.

Q6. (Scenario-Based) You are processing a 10GB log file on a machine with only 4GB of RAM. What is the correct way to handle the I/O?

A) Read the entire file into a `String` variable.

B) Use a `BinarySearch` on the file directly.

C) Stream the file line-by-line (using a Buffer).

D) Increase the virtual memory of the OS.

> Streaming (using `readLine()` or a generator) only keeps a small chunk of data in RAM at any time. This allows you to process files of any size without running out of memory.

Q7. (Conceptual) What does the "Flush" operation do in an I/O stream?

A) Deletes the contents of the file.

B) Forces any data currently sitting in the RAM buffer to be written to the underlying stream/disk immediately.

C) Closes the file connection.

D) Checks the file for viruses.

> Flushing is the manual way to ensure data integrity without closing the file. It is often used in logging or network sockets to ensure the recipient gets the data immediately.

Q8. (Code-Tracing) Trace the output in a language like Java or Python:

```
print("Apple", end="-")
print("Orange")
```

A) Apple
Orange

B) AppleOrange

C) Apple-Orange

D) Apple - Orange

> By default, `print` adds a newline (`\n`). Overriding the `end` parameter replaces the newline with the specified string, joining the two outputs on one line.

Q9. (Conceptual) What is "Object Serialization"?

A) Sorting a list of objects alphabetically.

B) Converting an object's state into a byte stream to be stored or transmitted.

C) Creating multiple instances of the same class.

D) Deleting an object from the heap.

> Serialization allows you to "freeze" an object and save it to a file, then "thaw" it (deserialization) later to restore the exact same object state.

Q10. (Scenario-Based) A program is stuck in a `while` loop reading from a network socket because no data is arriving. This state is known as:

A) Deadlock

B) Blocking I/O

C) Infinite Recursion

D) Segmentation Fault

> In "Blocking" I/O, the program execution stops at the read command and waits until data is available. To prevent this, developers use "Non-blocking I/O" or "Asynchronous" programming.

Deep Dive: Efficient I/O

1. Why use Buffering?

Disk I/O is thousands of times slower than RAM. If you write 1 byte to the disk 1000 times, the "arm" of the hard drive has to move 1000 times. With a **Buffer**, the program writes 1000 bytes to RAM (instant) and then moves the disk arm once to write the whole block (efficient).

2. The "Scanner" Trap in Competitive Programming

In Java, `Scanner(System.in)` uses Regex to parse input, which is very slow. For Graduate-level competitive programming or high-performance systems, `StringTokenizer` with `BufferedReader` is preferred to avoid performance bottlenecks.

Stable Web Resources

- Oracle: Basic I/O Tutorial
- Wikipedia: Standard Streams

Quick Revision Cheat Sheet

- **stdin (0):** Input. **stdout (1):** Output. **stderr (2):** Error.
- **"w":** Overwrite. **"a":** Append. **"r":** Read.
- **flush():** RAM → Disk (Now!).
- **close():** Frees the file handle. Essential for long-running apps.
- **Serialization:** Object → Bytes.
- **Blocking:** Program waits for data. **Non-blocking:** Program continues if no data.

END OF SUBTOPIC 06 — MOVING TO SUBTOPIC 07

Subtopic 07: Strings & Text Processing (15 MCQs)

Quick Notes

- **Immutability:** In languages like Java, Python, and C#, Strings cannot be changed. Any "modification" (like `s = s + "!"`) actually creates a brand new string object.
- **String Interning (Pool):** To save memory, identical string literals are stored in a special "String Pool." Multiple variables can point to the same physical memory address of "Hello."
- **Memory Efficiency:** For repeated concatenations (like in a loop), use `StringBuilder` or `StringBuffer`. This avoids creating thousands of temporary garbage objects.
- **Encoding:** 1) **ASCII:** 1 byte, 128 characters. 2) **Unicode/UTF-8:** Variable length (1-4 bytes), supports all global languages.
- **Regular Expressions (Regex):** A syntax for defining search patterns (e.g., `[a-z]+` for finding words).

Common Traps & Confusions

- **== vs .equals():** == checks if two variables point to the *same memory address*. `.equals()` checks if the *text content* is the same. Always use `.equals()` for string logic.
- **Length vs. Bytes:** A string with 5 characters might take 5 bytes in ASCII, but could take 10 or 15 bytes in UTF-8/UTF-16 depending on the characters (like emojis).
- **String Concatenation in a Loop:** `s += i` inside a loop is `O(N^2)` complexity because it copies the entire string in every iteration. Use a builder for `O(N)` performance.

MCQs

Q1. (Conceptual) What is the primary reason why Strings are designed to be **Immutable** in many high-level languages?

A) To make them run faster in loops.

B) Because computers cannot change binary text.

C) To allow String Pooling, improve security, and ensure thread-safety.

D) To prevent users from editing their usernames.

> Because strings are immutable, the system knows they won't change. This makes them safe to share across multiple threads and safe to use as keys in a Hash Map without the risk of the "key" changing its value.

Q2. (Code-Tracing) Trace the value of `s1` and `s2` in this Java/C# snippet:

```
String s1 = "Hello";
String s2 = s1;
s1 = s1 + " World";
print(s2);
```

A) Hello World

B) Hello

C) Error

D) World

> Strings are immutable. `s1 + " World"` creates a *new* string. `s1` now points to that new object, but `s2` still points to the original "Hello" object.

Q3. (Scenario-Based) You need to build a long HTML report by joining 10,000 small strings together in a loop. Which approach is the most efficient?

A) Use the + operator.

B) Use `String.concat()`.

C) Use a `StringBuilder` or `StringBuffer`.

D) Store the strings in a database first.

> The + operator creates a new object and copies the old content in every iteration (O(N^2)). `StringBuilder` uses a resizable internal array, making the total operation O(N), which is thousands of times faster for large loops.

Q4. (Debugging) A developer is checking if a user entered the correct password:

```
if (input_pass == "admin123") { ... }
```

Why is this logic flawed in languages like Java?

A) Password strings must be in double quotes.

B) == compares memory addresses; if the input was dynamically generated, the addresses won't match even if the text is identical.

C) "admin123" is too short for a string comparison.

D) Strings cannot be checked inside an `if` statement.

> In Java, == checks for "Reference Equality" (are they the same object?). To check "Value Equality" (is the text the same?), you must use `input_pass.equals("admin123")`.

Q5. (Conceptual) In UTF-8 encoding, how many bytes are typically used to represent a standard English character versus an Emoji?

A) 1 byte for both.

B) 2 bytes for both.

C) 1 byte for English; up to 4 bytes for Emoji.

D) English characters cannot be stored in UTF-8.

> UTF-8 is a variable-width encoding. It is backward compatible with ASCII (1 byte), but uses more bytes (up to 4) for complex characters and symbols from the Unicode set.

Q6. (Code-Tracing) Trace the output of this Python-style snippet:

```
text = "Graduate"
print(text[1:4])
```

A) Grad

B) rad

C) adu

D) Gra

> Slicing `[1:4]` starts at index 1 ('r') and goes *up to but not including* index 4 ('u'). The characters are index 1('r'), 2('a'), 3('d'). Result: "rad".

Q7. (Scenario-Based) You need to validate if a string is a valid email address. Which tool is best for this pattern matching task?

A) String Tokenizer

B) Regular Expressions (Regex)

C) `substring()` and `indexOf()`

D) A for loop checking every character

> Regex is the standard industry tool for complex pattern matching. A single regex can check for the presence of `@`, a domain name, and a valid TLD (like .com).

Q8. (Conceptual) What is the result of the "Interning" process in string management?

A) All strings are converted to lowercase.

B) Strings are encrypted for safety.

C) Identical string literals share a single memory location in the String Pool.

D) Strings are deleted when they are not in use.

> Interning ensures that if you use the word "System" 1,000 times in your code, only one copy of that word exists in a special memory area, saving RAM.

Q9. (Code-Tracing) What is the output?

```
String val = "10";
val += 20;
print(val);
```

A) 30

B) 1020

C) Error

D) 10

> Because `val` is a string, the + operator performs **Concatenation**, not addition. The integer 20 is converted to the string "20" and joined.

Q10. (Scenario-Based) A developer is processing a CSV file. For each line, they need to extract the "Name" which is between the first and second comma. Which function combination is most logical?

A) `reverse()` and `sort()`

B) `split(",")` or `indexOf()` with `substring()`

C) `toUpperCase()` and `trim()`

D) `replace()` and `append()`

> `split(",")` creates an array of all items, making the name easy to access by index. `substring()` allows you to cut a specific portion of the string based on index positions found by `indexOf()`.

Q11. (Conceptual) Which of the following is true about a "Null" string versus an "Empty" string?

A) They are exactly the same.

B) A Null string points to no object; an Empty string is a valid object with length 0.

C) An Empty string causes a NullPointerException.

D) Null strings take up more memory.

> `String s = null;` means the variable doesn't point to any object. `String s = "";` means the variable points to a real string object that just happens to contain zero characters.

Q12. (Calculation) If an algorithm to reverse a string of length `N` uses a simple loop to swap characters, what is its **Time Complexity**?

A) O(1)

B) O(log N)

C) O(N)

D) $O(N^2)$

> To reverse a string, you must visit every character at least once (or half of them). This results in a Linear growth rate relative to the string's length.

Q13. (Debugging) Look at this Regex: `^\d{5}$`. Which string will it match?

A) ABCDE

B) 1234

C) 54321

D) 123456

> Regex breakdown: `^` (start), `\d` (digit), `{5}` (exactly five times), `$` (end). Only a string of exactly five digits will match.

Q14. (Conceptual) What is the purpose of the `trim()` function in string processing?

A) To cut the string to a specific length.

B) To remove leading and trailing whitespace (spaces, tabs, newlines).

C) To delete vowels from the string.

D) To convert a string to a number.

> `trim()` is essential for sanitizing user input (e.g., if a user accidentally types a space after their username).

Q15. (Scenario-Based) You have a multithreaded application where 10 threads are all appending data to a shared log string simultaneously. Which class should you use in Java to ensure no data is corrupted?

A) StringBuilder

B) StringBuffer

C) String

D) char[]

> `StringBuilder` is faster but **not** thread-safe. `StringBuffer` is synchronized, meaning it includes internal locks to prevent multiple threads from corrupting the string data during simultaneous access.

Deep Dive: The String Pool

1. Why Pool?

Strings are the most commonly used objects in software. If every "Yes" and "No" literal in your code was a separate object, an app would waste megabytes of RAM. By making strings **Immutable**, the compiler can safely point 1,000 variables to the same single instance of "Yes" in the pool.

2. The Concatenation Trap

In `String result = a + b + c;`, the compiler actually optimizes this by using a `StringBuilder` internally. However, it **cannot** optimize a loop:

```
for (int i=0; i<100; i++) s += i;
```

In the loop above, the compiler creates a **new** StringBuilder and a **new** String in every single iteration. This is why manual use of StringBuilder is mandatory for loops.

Stable Web Resources

- Oracle: String API Documentation
- Regular-Expressions.info: Comprehensive Regex Guide

Quick Revision Cheat Sheet

- **Immutable:** Strings cannot change. (Java, C#, Python).
- **Pool:** A memory area where literals are cached for reuse.
- **==:** Address check. **.equals():** Content check.
- **StringBuilder:** Use for loops or many concatenations (O(N)).
- **UTF-8:** The standard for web and global text.
- **Regex:** `\d`=digit, `\w`=word, +=one or more.
- **Trim:** Cleans up " spaces ".

END OF SUBTOPIC 07 — MOVING TO SUBTOPIC 08

Subtopic 08: Arrays & Collections (20 MCQs)

Quick Notes

- **Arrays:** Fixed-size, contiguous memory blocks. Accessing an element by index is `O(1)`, but inserting/deleting in the middle is `O(N)` due to shifting.
- **Dynamic Arrays (ArrayList/Vector):** Resizable arrays that grow automatically. They offer fast random access but slow middle-insertions.
- **Linked Lists:** Elements (nodes) are scattered in memory and linked by pointers. Fast insertions/deletions `O(1)` if the position is known, but slow access `O(N)`.
- **Sets:** Collections that allow **no duplicate elements**. `HashSet` is unordered (O(1) search); `TreeSet` is sorted (O(log N) search).
- **Maps (Dictionaries):** Key-Value pairs. Keys must be unique. `HashMap` uses hashing for near-instant `O(1)` lookups.
- **Stacks & Queues: Stack** is LIFO (Last-In-First-Out). **Queue** is FIFO (First-In-First-Out).

Common Traps & Confusions

- **Array vs. ArrayList:** Arrays have a fixed `.length`; ArrayLists have a dynamic `.size()`. Arrays can hold primitives; many Collections only store Objects.
- **Concurrent Modification:** Modifying a collection (adding/removing) while iterating through it using a `for-each` loop usually throws an exception.
- **The "Contains" Trap:** Checking `list.contains()` is `O(N)` because it scans the whole list. Checking `set.contains()` is `O(1)`. Always use a Set for uniqueness checks.

MCQs

Q1. (Conceptual) Why is accessing an element in a standard Array by its index considered a Constant Time `O(1)` operation?

A) The computer searches through the array until it finds the index.

B) Because memory is contiguous, the address of any element can be calculated directly using a mathematical formula (Base + Index * Size).

C) Arrays use a hashing algorithm to find elements.

D) Arrays are always stored in the CPU cache.

> Since arrays are stored in a continuous block of memory, the CPU doesn't "search." It simply calculates: `Start_Address + (Index * Data_Type_Size)` and jumps straight to that byte.

Q2. (Scenario-Based) You are given a list of 1 million student IDs, and you need to find all the **unique** IDs in the list. Which data structure is the most efficient choice for this task?

A) ArrayList

B) Linked List

C) HashSet

D) Stack

> A `HashSet` automatically ignores duplicates. Because its lookup and insertion time is `O(1)`, it can process the million IDs much faster than a list, which would require `O(N^2)` to check every element against every other element.

Q3. (Code-Tracing) Trace the output of this logic involving a Dynamic Array (ArrayList):

```
list = [10, 20, 30]
list.add(1, 15)
list.remove(2)
print(list)
```

A) [10, 15, 30]

B) [10, 15, 30]

C) [15, 20, 30]

D) [10, 20, 15]

> 1. Start: `[10, 20, 30]`.
> 2. `add(1, 15)`: Inserts 15 at index 1, shifting others → `[10, 15, 20, 30]`.
> 3. `remove(2)`: Removes the item at index 2 (which is 20) → `[10, 15, 30]`.

Q4. (Calculation) In a 2D integer array `matrix[5][10]`, how many total elements are stored in memory, and what is the index of the very last element?

A) 50 elements, index [5][10]

B) 15 elements, index [4][9]

C) 50 elements, index [4][9]

D) 50 elements, index [5][9]

> Total elements = 5 * 10 = 50. Since indices are 0-based, the rows go from 0-4 and the columns go from 0-9. The last element is `[4][9]`.

Q5. (Comparison) What is a major performance disadvantage of a **Linked List** compared to an **Array**?

A) Insertions at the beginning are slow.

B) It uses less memory.

C) It does not support random access; you must traverse nodes one by one to find an element.

D) It cannot store objects.

> In an array, you can jump to `arr[500]` instantly. In a Linked List, you must start at the "Head" and follow 500 pointers to reach that element, making search `O(N)`.

Q6. (Debugging) A developer is iterating through a List and removing items that meet a condition:

```
for (Item x : myList) {
    if (x.isExpired()) myList.remove(x);
}
```

What is the likely outcome in Java or C#?

A) The expired items are removed correctly.

B) A ConcurrentModificationException is thrown.

C) The loop skips every second item.

D) The program enters an infinite loop.

> You cannot modify a collection directly while using a fail-fast iterator (the hidden mechanism behind the for-each loop). To remove items during iteration, you must use an explicit `Iterator` and call `it.remove()`.

Q7. (Conceptual) Which data structure follows the **LIFO** (Last-In-First-Out) principle?

A) Queue

B) Stack

C) Array

D) Map

> A Stack works like a pile of plates. The last one you put on top is the first one you take off.

Q8. (Scenario-Based) You are building an "Undo" feature for a text editor. Every time the user types, the change is saved. When they press "Undo," the **most recent** change is reversed. Which collection is most appropriate?

A) Queue

B) Set

C) Stack

D) HashMap

> "Undo" requires getting the most recent action back first. This is exactly what a Stack's `pop()` operation does.

Q9. (Code-Tracing) What is the output?

```
int[] arr = new int[3];
print(arr[0]);
```

(In a language like Java where arrays are zero-initialized)

A) Garbage Value

B) Null

C) 0

D) Error

> In Java and C#, numeric arrays are automatically initialized with 0 for all elements upon creation. In C++, this would result in a "Garbage Value" (whatever was previously in that memory).

Q10. (Comparison) What is the **Time Complexity** of looking up a value by its **Key** in a `HashMap` (on average)?

A) O(N)

B) O(log N)

C) O(1)

D) O(N log N)

> A HashMap uses a hash function to compute an index for the key. This allows the computer to go directly to the value's location, making it near-instant regardless of how many items are in the map.

Q11. (Scenario-Based) You are managing a printer queue where the first document sent must be the first one printed. Which data structure is best?

A) Stack

B) Queue

C) HashSet

D) Array

> A Queue follows FIFO (First-In-First-Out), making it perfect for scheduling tasks in the order they arrived.

Q12. (Conceptual) What happens when a **Hash Collision** occurs in a HashMap?

A) The program crashes.

B) The old value is overwritten.

C) Multiple keys result in the same hash index, and the Map must handle them (e.g., using a linked list at that index).

D) The HashMap automatically doubles its size.

> Collisions are inevitable. When two different keys produce the same hash, the map typically stores them in a "bucket" (using a list or tree) and then differentiates them using the `.equals()` method.

Q13. (Code-Tracing) Trace the value of `result`:

```
Stack s = new Stack();
s.push(10);
s.push(20);
s.pop();
s.push(30);
result = s.peek();
```

A) 10

B) 20

C) 30

D) 60

> 1. Push 10 → [10]
> 2. Push 20 → [10, 20]
> 3. Pop → [10] (20 removed)
> 4. Push 30 → [10, 30]
> 5. Peek → returns the top element without removing it: 30.

Q14. (Calculation) What is the **Time Complexity** of inserting a new element at the **index 0** of an `ArrayList` of size `N`?

A) O(1)

B) O(N)

C) O(log N)

D) O(N^2)

> To put something at the very beginning of an array, every single existing element must be shifted one position to the right to make room. If there are N elements, this takes N operations.

Q15. (Scenario-Based) A developer needs to store pairs of "Country Name" and "Capital City." They need to be able to quickly find the capital given a country name. Which collection is best?

A) List

B) Set

C) Map

D) Queue

> A Map is designed for Key-Value pairs. The Country Name would be the Key, and the Capital would be the Value.

Q16. (Conceptual) What is the difference between a `Capacity` and `Size` in a dynamic array like `ArrayList`?

A) They are the same.

B) Capacity is how much memory is allocated; Size is how many elements are currently in it.

C) Size is measured in bits; Capacity is in bytes.

D) Capacity is for primitives; Size is for objects.

> An ArrayList might allocate space for 10 items (Capacity) but only contain 3 actual items (Size). Once Size reaches Capacity, the array is resized.

Q17. (Debugging) Why is using an `Array` usually safer for performance-critical code than an `ArrayList` of Objects?

A) Arrays cannot have bugs.

B) Arrays of primitives avoid "Boxing/Unboxing" overhead and provide better "Cache Locality."

C) Arrays are easier to read.

D) ArrayLists do not support loops.

> ArrayLists store Objects. Storing an `int` in a list requires "Auto-boxing" it into an `Integer` object, which consumes more memory and CPU. Primitives in a raw array are packed tightly together, which the CPU can process extremely fast.

Q18. (Scenario-Based) You need to implement a "Priority Task List" where the task with the **highest importance** is always finished first, regardless of when it was added. Which structure is best?

A) Simple Queue

B) Stack

C) PriorityQueue (Heap)

D) LinkedList

> A `PriorityQueue` doesn't follow the order of arrival. Instead, it uses a sorting criteria (like a Heap) to ensure the "most important" item is always at the head of the queue.

Q19. (Code-Tracing) Trace the output:

```
set = new HashSet();
set.add("A");
set.add("B");
set.add("A");
print(set.size());
```

A) 3

B) 2

C) 1

D) Error

> A `Set` cannot contain duplicate values. Adding "A" a second time does nothing. The set only contains `{"A", "B"}`, so the size is 2.

Q20. (Conceptual) What is the "Load Factor" of a HashMap?

A) The weight of the hash function.

B) The threshold (e.g., 0.75) at which the Map will automatically increase its capacity to avoid collisions.

C) The number of collisions that have occurred.

D) The time it takes to hash a key.

> If the load factor is 0.75, the HashMap will resize and rehash all elements once it is 75% full. This keeps the Map sparse enough to maintain `O(1)` performance.

Deep Dive: Memory Layout & Performance

1. Contiguous Memory vs. Pointers

Arrays are **Cache-Friendly**. Because the data is side-by-side, the CPU can load an entire chunk of the array into its high-speed Cache at once. **Linked Lists** are not

cache-friendly; because nodes can be anywhere in RAM, the CPU has to "fetch" from different addresses constantly, which is much slower.

2. The Cost of Resizing

When an `ArrayList` gets full, it doesn't just add one slot. It usually creates a **new array** that is double the size and **copies** all elements over. This is an `O(N)` operation. To optimize, if you know you will store 1,000 items, initialize the list with `new ArrayList(1000)` to avoid repeated resizing.

Stable Web Resources

- Oracle: The Collection Framework
- Wikipedia: Data Structures Overview
- VisuAlgo: Linked List & Array Visualization

Quick Revision Cheat Sheet

Operation	Array / ArrayList	Linked List	HashMap / Set
Access (by Index)	O(1) - Fast	O(N) - Slow	N/A
Search (by Value)	O(N)	O(N)	O(1) - Fastest
Insert at Start	O(N) - Bad	O(1) - Good	O(1)
Memory Usage	Low	High (Pointers)	High (Hash table)

- **Stack:** LIFO (Undo button, Call stack).
- **Queue:** FIFO (Printer, Customer line).
- **Set:** No duplicates.
- **Map:** Key-Value pairs.

END OF SUBTOPIC 08 — MOVING TO SUBTOPIC 09

Subtopic 09: Object-Oriented Programming (25 MCQs)

Quick Notes

- **The Four Pillars:** 1) **Encapsulation:** Hiding internal state via access modifiers (private/protected). 2) **Inheritance:** Reusing code by creating a parent-child relationship. 3) **Polymorphism:** One interface, multiple forms (Overloading/Overriding). 4) **Abstraction:** Hiding complexity and showing only essential functionality.
- **Classes vs. Objects:** A Class is the blueprint (stored in Method Area). An Object is the instance (stored on the Heap).
- **Constructors:** Special methods called during object creation. They have no return type.
- **Interfaces vs. Abstract Classes:** Interfaces define a *contract* (What to do). Abstract Classes define a *base identity* (What it is).
- **The "Is-A" vs. "Has-A" Relationship:** Inheritance represents "Is-A" (A Dog is an Animal). Composition represents "Has-A" (A Car has an Engine).

Common Traps & Confusions

- **The Diamond Problem:** In languages with Multiple Inheritance (like C++), if a class inherits from two parents who both have the same method, the compiler gets confused. Java/C# solve this by only allowing Multiple Inheritance of *Interfaces*.
- **Static vs. Instance:** Static methods cannot access instance variables (non-static) because static methods exist before any object is created.
- **Overloading vs. Overriding:** Overloading is *Compile-time* (same name, different params). Overriding is *Runtime* (same name/params, different class levels).

MCQs

Q1. (Conceptual) Which OOP principle is primarily concerned with reducing "Tight Coupling" between different parts of a system?

A) Inheritance

B) Overloading

C) Encapsulation

D) Instantiation

> Encapsulation hides the internal implementation of a class. By exposing only a public API (methods), other classes don't depend on *how* data is stored, only on the methods provided, which reduces coupling.

Q2. (Scenario-Based) You have a class `Shape` with a method `draw()`. You have subclasses `Circle` and `Square` that provide their own versions of `draw()`. At runtime, the program decides which `draw()` to call based on the object type. This is:

A) Static Binding

B) Dynamic (Late) Binding

C) Encapsulation

D) Method Overloading

> This is a core part of Polymorphism. Because the decision of which method to execute happens at **Runtime** (based on the actual object on the heap), it is called Dynamic or Late Binding.

Q3. (Code-Tracing) Look at this Java/C# snippet. What is the output?

```
class Parent {
    void show() { print("Parent"); }
}
class Child extends Parent {
    void show() { print("Child"); }
}
Parent obj = new Child();
obj.show();
```

A) Parent

B) Child

C) ParentChild

D) Compilation Error

> Even though the *Reference* is of type Parent, the *Object* is a Child. In method overriding, the version in the actual object type (Child) always takes priority at runtime.

Q4. (Conceptual) What is the "Diamond Problem" in Object-Oriented Programming?

A) When a class has too many attributes.

B) When an object is deleted twice from memory.

C) Ambiguity arising when a class inherits from two classes that both inherit from a single superclass.

D) A syntax error caused by using too many interfaces.

> If Class D inherits from B and C, and both B and C inherit from A, D might receive two conflicting versions of a method from A. This is why Java and C# do not support multiple inheritance of classes.

Q5. (Scenario-Based) A developer wants to create a class `Vehicle` that can never be instantiated on its own but provides a common `startEngine()` method for `Car` and `Truck`. What should `Vehicle` be?

A) A Private Class

B) A Static Class

C) An Abstract Class

D) An Interface

> Abstract classes are meant to be base classes. They cannot be turned into objects (instantiated) using the `new` keyword, forcing developers to inherit from them instead.

Q6. (Code-Tracing) Trace the value of `count`:

```
class Counter {
    static int count = 0;
    Counter() { count++; }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
```

```
Counter c3 = new Counter();
print(Counter.count);
```

A) 1

B) 0

C) 3

D) Error

> The `static` keyword means `count` is shared by **all** instances. Every time the constructor runs (3 times), it increments the same shared variable.

Q7. (Comparison) What is a key difference between an **Interface** and an **Abstract Class** in modern Java (post-Java 8)?

A) Abstract classes can have methods; interfaces cannot.

B) A class can implement multiple Interfaces but can only extend one Abstract Class.

C) Interfaces cannot have any logic.

D) Abstract classes are faster than interfaces.

> This is the primary architectural difference. Inheritance is limited to one parent (Single Inheritance), but a class can satisfy multiple "contracts" by implementing many interfaces.

Q8. (Conceptual) Which of the following best describes **Composition** over Inheritance?

A) Using a child class to access parent methods.

B) Building complex objects by combining simpler objects as attributes (Has-A).

C) Copying the code from one class to another.

D) Deleting a class to save memory.

> Composition (e.g., a `Car` class having an `Engine` object as a field) is often more flexible than inheritance because it allows you to change behavior at runtime by swapping objects.

Q9. (Debugging) Why will the following code fail to compile?

```
class Test {
    int x = 10;
    static void display() {
        print(x);
```

```
    }
}
```

A) x must be private.

B) A static method cannot access a non-static (instance) variable.

C) `display()` must return an int.

D) `print` is not a valid keyword.

> Static methods exist at the class level. They don't know which specific object's x they should print, because they can be called even if zero objects of `Test` have been created.

Q10. (Conceptual) What is the purpose of a **Constructor**?

A) To delete an object from memory.

B) To initialize the state (attributes) of an object when it is created.

C) To make the class run faster.

D) To convert a class into an interface.

> Constructors are called automatically when new is used. Their job is to set the initial values of the object's variables.

Q11. (Scenario-Based) You are designing a `PaymentSystem`. You have different types of payments: `CreditCard`, `PayPal`, and `Crypto`. They all must have a `process()` method. What is the best way to enforce this?

A) Copy the `process()` method into each class.

B) Create an interface `IPayment` with a `process()` method and make all classes implement it.

C) Use a global function that checks the payment type with `if-else`.

D) Make all payment types inherit from `CreditCard`.

> An Interface acts as a contract. It guarantees that any class claiming to be a "Payment" **must** have a `process()` method, allowing you to handle them all uniformly.

Q12. (Code-Tracing) What is **Method Overloading**?

A) Changing the logic of a parent method in a child class.

B) Defining multiple methods in the same class with the same name but different parameters.

C) Calling a method too many times in a loop.

D) Deleting a method.

> Overloading allows a class to have, for example, `add(int a, int b)` and `add(double a, double b)`. The compiler picks the right one based on the arguments you pass.

Q13. (Conceptual) What is the meaning of the `final` keyword (or `sealed` in C#) when applied to a **Class**?

A) The class cannot be instantiated.

B) The class cannot be inherited from (it cannot have children).

C) All methods in the class are static.

D) The class is deleted when the program ends.

> Marking a class `final` prevents other developers from extending it. This is often done for security or to ensure the class's behavior isn't modified (e.g., the `String` class in Java is final).

Q14. (Scenario-Based) In a `User` class, the `password` field is set to `private` and can only be changed via a `setPassword()` method that checks for a minimum length. Which OOP principle is being applied?

A) Inheritance

B) Encapsulation

C) Diversity

D) Runtime Polymorphism

> This is "Data Hiding." By making the field private and providing a public method, you control how the data is accessed and validated, protecting the integrity of the object.

Q15. (Code-Tracing) What will happen if a class does not define a constructor?

A) The program will not compile.

B) No objects of that class can be created.

C) The compiler provides a hidden "Default Constructor" with no parameters.

D) The object will have no memory.

> Most OOP languages provide a default constructor automatically so you can still do `new MyClass()`. However, if you define *any* custom constructor, the default one is removed.

Q16. (Conceptual) What does the `super` (or `base`) keyword do inside a child class constructor?

A) It makes the child class run faster.

B) It calls the constructor of the Parent class.

C) It deletes the parent class.

D) It converts the class to an interface.

> Before a child object is fully created, its parent "part" must be initialized. `super()` ensures the parent's constructor runs first.

Q17. (Scenario-Based) You have a list of `Employee` objects. Some are `Managers`, some are `Developers`. You want to calculate the total salary by calling `emp.getSalary()` on every item in the list. This is possible because of:

A) Encapsulation

B) Data Hiding

C) Polymorphism

D) Multiple Inheritance

> Polymorphism allows you to treat a group of different objects (Managers, Developers) as their common type (Employee) and call the same method name, while getting the specific behavior for each.

Q18. (Comparison) What is the difference between **Aggregation** and **Composition**?

A) There is no difference.

B) Composition is "Is-A"; Aggregation is "Has-A".

C) In Composition, the child object cannot exist without the parent; in Aggregation, the child can exist independently.

D) Aggregation is faster.

> Example: A **Room** is part of a **House** (Composition - if house is deleted, room is gone). A **Teacher** is part of a **Department** (Aggregation - if department is deleted, teacher still exists).

Q19. (Debugging) What is a **Memory Leak** in an OOP language like C++?

A) When an object takes too much RAM.

B) When an object is created on the heap using 'new' but is never 'deleted', leaving

unreachable memory.

C) When a variable is used twice.

D) When the constructor fails.

> In unmanaged languages, if you lose the pointer to an object on the heap without freeing it, that memory is "lost" to the system until the program restarts.

Q20. (Conceptual) What is a **Virtual Method Table (V-Table)** used for?

A) To sort objects in a list.

B) To support Dynamic Dispatch/Polymorphism by looking up the correct method address at runtime.

C) To store private variables.

D) To speed up the compilation process.

> V-Tables are the internal mechanism that makes polymorphism work. Every object with overridden methods has a hidden table of pointers to the correct versions of those methods.

Q21. (Code-Tracing) If a class `A` has a `protected` member `x`, who can access it?

A) Only class A.

B) Everyone.

C) Class A, its subclasses, and classes in the same package.

D) Only classes in different packages.

> `protected` is less restrictive than `private` but more restrictive than `public`. Its main purpose is to allow child classes to access parent data.

Q22. (Conceptual) What is the **Single Responsibility Principle** (the S in SOLID)?

A) A class should only have one object.

B) A class should only have one method.

C) A class should have only one reason to change (it should only do one job).

D) A class should only be written by one developer.

> If a class handles both "Database Logic" and "UI Printing," it has two responsibilities. Changing the UI could accidentally break the Database part. SOLID design says they should be separate.

Q23. (Scenario-Based) You want to prevent a method in a parent class from being **overridden** by any child class. What keyword should you use?

A) static

B) abstract

C) final (or sealed)

D) private

> Marking a method `final` ensures that its implementation is "final" and cannot be changed by subclasses. (Note: `private` methods are also effectively final).

Q24. (Conceptual) What is a **Copy Constructor**?

A) A constructor that copies a class file.

B) A constructor that creates a new object as a copy of an existing object.

C) A constructor that uses the 'super' keyword.

D) A constructor that returns a string.

> Copy constructors (common in C++) allow you to create `Object B` using `Object A`'s data, ensuring a "deep copy" where all values are duplicated.

Q25. (Code-Tracing) In a language like Python, what is the purpose of the `self` (or `this` in Java) keyword?

A) To refer to the parent class.

B) To refer to the current instance of the object.

C) To create a new object.

D) To make the code run faster.

> When a method is called, the computer needs to know *which* object's data to use. `this` or `self` acts as a pointer to the specific object that called the method.

Deep Dive: The Architecture of OOP

1. The V-Table and Dynamic Dispatch (The "How" of Polymorphism)

When you call `obj.draw()` in a polymorphic system, the compiler doesn't actually know which code to run. Instead, it generates code that follows a three-step process:

1. Follow the **Object Reference** to the object on the Heap.
2. Look at the object's hidden **V-Table Pointer** (vptr).
3. Look up the address for `draw()` in that object's specific **Virtual Method Table**.

This is why polymorphism has a tiny "performance hit" compared to direct function calls—it requires a few extra memory lookups.

2. Interfaces vs. Abstract Classes (The Deep Choice)

Feature	Interface	Abstract Class
Intent	Defines "Can-Do" capability (e.g., `IDrawable`).	Defines "Is-A" identity (e.g., `Vehicle`).
State	Cannot have instance variables (prior to Java 8).	Can have instance variables (fields).
Multiplicity	Multiple implementation allowed.	Single inheritance only.
Usage	Decoupling unrelated classes (e.g., a `Bird` and a `Plane` both implement `IFlyable`).	Sharing code between closely related classes (e.g., `Car` and `Truck`).

3. The SOLID Principles for Graduates

Professional OOP isn't just about classes; it's about **Scalable Design**:

- **S: Single Responsibility:** A class should do one thing.
- **O: Open/Closed:** Classes should be open for extension (inheritance) but closed for modification (don't change working code).
- **L: Liskov Substitution:** A child object should be able to replace a parent object without breaking the app.
- **I: Interface Segregation:** Don't force a class to implement methods it doesn't need. Split big interfaces into small ones.

- **D: Dependency Inversion:** Depend on abstractions (interfaces), not on concrete classes.

Stable Web Resources

- Wikipedia: Object-Oriented Programming
- GeeksforGeeks: SOLID Principles Explained
- Oracle: OOP Concepts Tutorial
- Wikipedia: Virtual Method Tables (V-Tables)

Quick Revision Cheat Sheet

- **Encapsulation:** Data hiding. Private fields + Public methods.
- **Inheritance:** Code reuse. Use for "Is-A" relationships.
- **Polymorphism:** Overriding (Runtime) vs. Overloading (Compile-time).
- **Abstract Class:** Can have state; can't be instantiated; single inheritance.
- **Interface:** Contract only; multiple implementations; highly decoupled.
- **Static:** Class-level, shared by all objects, no access to `this`.
- **Constructor:** No return type; initializes object state.
- **Composition:** "Has-A" relationship. Generally more flexible than inheritance.

END OF SUBTOPIC 09 — MOVING TO SUBTOPIC 10

Subtopic 10: Memory Management Concepts (10 MCQs)

Quick Notes

- **Memory Segments:** A program's memory is divided into: 1) **Text:** Executable instructions. 2) **Data:** Global/Static variables. 3) **Stack:** Local variables and function calls. 4) **Heap:** Dynamically allocated objects.

- **Manual Management:** In languages like C/C++, the developer uses `malloc/free` or `new/delete`. Error-prone but high performance.
- **Automatic Management (Garbage Collection):** In Java/Python/C#, a background process identifies and deletes objects that are no longer reachable from the "Root."
- **Pointers vs. References:** A **Pointer** stores a raw memory address and can be manipulated (pointer arithmetic). A **Reference** is an alias for an existing object and is usually safer.
- **RAII (Resource Acquisition Is Initialization):** A C++ technique where a resource (memory, file handle) is tied to an object's lifetime. When the object goes out of scope, the resource is automatically released.

Common Traps & Confusions

- **Dangling Pointer:** A pointer that points to a memory location that has already been deleted/freed. Accessing it causes unpredictable crashes.
- **Memory Leak:** Memory that is allocated on the Heap but never freed, even though the program no longer has a pointer to it. This "leaked" memory cannot be reused until the program ends.
- **Stack Overflow vs. Heap Overflow:** Stack Overflow usually results from infinite recursion. Heap Overflow (or OutOfMemoryError) happens when you create too many large objects.

MCQs

Q1. (Conceptual) In the standard memory layout of a C/C++ program, which segment grows **downward** (towards lower addresses) during function calls?

A) Heap

B) Data Segment

C) Stack

D) Text Segment

> In most architectures, the Stack grows downward from high memory addresses, while the Heap grows upward from low addresses. This "growing towards each other" design maximizes the use of available space.

Q2. (Scenario-Based) A Java application is running, and the RAM usage is steadily increasing. Suddenly, the usage drops by 500MB without any user action. What most likely occurred?

A) The CPU became faster.

B) A Memory Leak was fixed automatically.

C) The Garbage Collector (GC) triggered a "Major Collection" cycle.

D) The program moved variables from the Heap to the Stack.

> Garbage Collection is non-deterministic; it runs when the JVM decides memory is low. A sudden drop in RAM usage indicates that the GC has successfully reclaimed memory from unreachable objects.

Q3. (Code-Tracing / C++) What is the danger in the following code snippet?

```
int* ptr = new int(10);
ptr = new int(20);
```

A) Dangling Pointer

B) Memory Leak

C) Segmentation Fault

D) Stack Overflow

> The first line allocates memory for an integer (10) on the heap. The second line makes `ptr` point to a *new* integer (20). The memory for "10" is still allocated, but we no longer have a pointer to it, so it can never be deleted. This is a memory leak.

Q4. (Comparison) What is the primary functional difference between `malloc()` in C and `new` in C++?

A) `malloc()` is faster.

B) `new` allocates memory AND calls the object's constructor; `malloc()` only allocates raw bytes.

C) `malloc()` handles objects; `new` handles primitives.

D) There is no difference.

> `new` is an operator that is "object-aware." It calculates the size needed and automatically initializes the object. `malloc()` is a function that returns a `void*` and knows nothing about the data type being stored.

Q5. (Conceptual) Which Garbage Collection algorithm works by keeping a count of how many pointers refer to an object, and deleting the object when the count reaches zero?

A) Mark-and-Sweep

B) Reference Counting

C) Generational Collection

D) Stop-the-world

> Reference Counting is simple but has a major flaw: it cannot handle "Circular References" (where Object A points to B, and B points to A, keeping both counts at 1 even if the rest of the program has forgotten them).

Q6. (Debugging) What typically causes a **Segmentation Fault** (SIGSEGV)?

A) Running out of RAM.

B) A logical error in a math formula.

C) Attempting to access a memory address that the program does not have permission to access (e.g., dereferencing a NULL pointer).

D) Using too many global variables.

> The OS protects memory. If your program tries to read or write to an address that hasn't been assigned to it (like address `0x0`), the OS kills the process with a Segmentation Fault to prevent system corruption.

Q7. (Conceptual) In modern Garbage Collectors, what is the "Generational Hypothesis"?

A) Older objects are more likely to be deleted.

B) Most objects "die young" (become unreachable quickly after creation).

C) Memory should be divided by the generation of the CPU.

D) Objects created by loops are permanent.

> Observation shows that most objects are temporary (local variables, temporary strings). GCs use this by dividing the heap into "Young" and "Old" generations, scanning the "Young" area much more frequently to improve performance.

Q8. (Scenario-Based) A C++ developer uses `std::unique_ptr` instead of raw pointers. This is an application of which memory management philosophy?

A) Manual Garbage Collection

B) Pointer Arithmetic

C) RAII (Resource Acquisition Is Initialization)

D) Static Allocation

> Smart pointers like `unique_ptr` use RAII. When the smart pointer object goes out of scope (e.g., a function ends), its destructor is called, which automatically deletes the heap memory it was managing.

Q9. (Calculation) If a 32-bit system has a maximum of 4GB of addressable memory, how many bits are used for each memory address?

A) 16 bits

B) 32 bits

C) 64 bits

D) 8 bits

> 2^32 equals approximately 4.29 billion. Therefore, a 32-bit address can uniquely identify 4.29 billion individual bytes (4GB). This is why 32-bit operating systems cannot use more than 4GB of RAM.

Q10. (Conceptual) What is **Memory Fragmentation**?

A) When memory is physically cracked.

B) When the CPU cache is full.

C) When free memory is broken into small, non-contiguous blocks, making it impossible to allocate a large continuous object even if total free space is sufficient.

D) When the Stack and Heap collide.

> Fragmentation occurs over time as objects of different sizes are allocated and deleted. "Compaction" is the process where a Garbage Collector moves objects around to group free space back into one large block.

Deep Dive: The Physical and Logical Memory Map

1. The Anatomy of a Process in RAM

When you run a program, the Operating System allocates a "Virtual Address Space." It is logically organized as follows:

Section	Content	Lifetime
Text Segment	The actual binary machine code (Read-only).	Entire program run.
Data Segment	Global variables and static variables initialized by the programmer.	Entire program run.
BSS Segment	Uninitialized global/static variables (set to zero by default).	Entire program run.
Stack	Function parameters, return addresses, and local variables.	Scope of the function.
Heap	Memory requested at runtime (`new`, `malloc`).	Until manually freed or Garbage Collected.

2. How "Mark-and-Sweep" Works (The GC standard)

Most modern GCs (Java, C#, Go) avoid the circular reference problem of Reference Counting by using **Mark-and-Sweep**:

1. **Mark:** The GC starts from "Roots" (global variables, active stack frames) and traverses every pointer, marking every object it can reach as "Live."
2. **Sweep:** The GC scans the entire heap. Any object NOT marked as "Live" is considered garbage and its memory is reclaimed.

3. Pointers vs. References (Graduate Distinction)

- **Pointers:** Can be NULL. Can be reassigned to point elsewhere. Can perform arithmetic (`ptr++`). Very powerful but dangerous (used in C/C++).
- **References:** Cannot be NULL (usually). Must be initialized when created. Cannot be reassigned to a different object. Much safer (used in Java, C#, and C++ references).

Stable Web Resources

- Wikipedia: Memory Management
- GeeksforGeeks: Memory Layout of C Programs
- Oracle: Garbage Collection Tuning Guide

Quick Revision Cheat Sheet

- **Stack:** Static, LIFO, very fast, local scope.
- **Heap:** Dynamic, manual/GC managed, large, flexible.
- **Memory Leak:** Memory allocated but "lost" (no pointer left).
- **Dangling Pointer:** Pointer points to deleted memory.
- **Garbage Collection:** Automates memory cleanup (Java, Python).
- **Reference Counting:** Deletes at zero refs (but fails at cycles).
- **Mark-and-Sweep:** Reaches from roots to find live objects.
- **Fragmentation:** Small gaps in memory that prevent large allocations.

END OF SUBTOPIC 10 — MOVING TO SUBTOPIC 11

Subtopic 11: Exception & Error Handling (15 MCQs)

Quick Notes

- **Exception vs. Error:** An **Exception** is an event the program can potentially handle (e.g., File Not Found). An **Error** is a serious system-level failure that the program usually cannot recover from (e.g., OutOfMemory, StackOverflow).
- **Try-Catch-Finally:** 1) `try`: Code that might throw an exception. 2) `catch`: Code to handle the exception. 3) `finally`: Code that **always** runs, regardless of whether an exception occurred.

- **Checked Exceptions:** Must be handled or declared (using `throws`) at compile-time (e.g., IOException in Java).
- **Unchecked (Runtime) Exceptions:** Programming errors that occur during execution (e.g., NullPointerException, ArrayIndexOutOfBounds). They do not require mandatory handling.
- **Throw vs. Throws:** `throw` is used to trigger an exception manually. `throws` is used in a method signature to warn the caller that an exception might occur.
- **Resource Management:** Using `try-with-resources` (Java) or `using` (C#) to automatically close files/sockets.

Common Traps & Confusions

- **Swallowing Exceptions:** An empty `catch` block. This is a major anti-pattern because it hides bugs, making them impossible to trace.
- **Catch Order:** You must catch the *most specific* exception first (e.g., `FileNotFoundException`) before the *most general* (e.g., `Exception`). Reversing this makes specific blocks unreachable.
- **Return in Finally:** If a `return` statement exists in both the `try` and `finally` blocks, the `finally` return will override the `try` return.

MCQs

Q1. (Conceptual) In the Java/C# exception hierarchy, which type of exception usually represents a programmer's logic error and does not need to be explicitly declared in the method signature?

A) Checked Exception

B) Unchecked (Runtime) Exception

C) System Error

D) Interrupted Exception

> Unchecked exceptions (like `NullPointerException` or `ArithmeticException`) are typically the result of bad logic. Since they can happen anywhere, the language doesn't force you to declare them, unlike Checked exceptions (I/O, SQL).

Q2. (Code-Tracing) What is the output of the following pseudocode?

```
function check():
    try:
        print("A")
        return 1
    catch:
        print("B")
    finally:
        print("C")
```

A) A, 1

B) A, B, C

C) A, C (followed by return 1)

D) A, B

> The `finally` block is guaranteed to run. Even if the `try` block encounters a `return`, the execution "pauses," runs the `finally` code, and then completes the return.

Q3. (Scenario-Based) A developer writes a `catch (Exception e) { }` block with no code inside. Why is this considered "Dangerous" in professional software development?

A) It makes the program run slower.

B) It "swallows" the exception, hiding the fact that an error occurred and making debugging impossible.

C) It causes a memory leak.

D) It prevents the `finally` block from running.

> Ignoring an exception means the program may continue in an unstable or corrupted state without any log or error message to help the developer find the root cause.

Q4. (Comparison) What is the primary difference between `throw` and `throws`?

A) They are identical.

B) `throw` is used to actually trigger an exception; `throws` is used in the method header to declare potential exceptions.

C) `throws` is for errors; `throw` is for exceptions.

D) `throw` is used in Python; `throws` is used in Java.

> `throw new Exception()` is an action. `void myMethod() throws IOException` is a declaration (a warning to the caller).

Q5. (Debugging) Look at this C++ catch sequence. Why is it problematic?

```
try { ... }
catch (std::exception& e) { ... }
catch (std::runtime_error& e) { ... }
```

A) You cannot have two catch blocks.

B) The second block is unreachable because `runtime_error` is a child of `exception`.

C) C++ doesn't support `std::exception`.

D) It will cause a compiler crash.

> Exceptions must be caught from "Most Specific" to "Most General." Since `runtime_error` is a type of `exception`, the first block will catch everything, leaving the second block useless.

Q6. (Conceptual) What happens if an exception is thrown inside a function, but there is no `try-catch` block to handle it in that function?

A) The program crashes immediately.

B) The exception "bubbles up" (propagates) to the calling function in the stack.

C) The exception is automatically ignored.

D) The computer restarts.

> This is "Exception Propagation." The runtime searches down the call stack for a handler. If it reaches the `main()` method and still finds no handler, only then does the program terminate.

Q7. (Scenario-Based) You are writing a banking app. A user tries to withdraw $100 but only has $50. Which is the most appropriate way to handle this?

A) Print "Error" and exit the program.

B) Return 0.

C) Throw a custom `InsufficientFundsException`.

D) Use a `while` loop to wait for more money.

> Custom exceptions make code more readable and allow the calling logic to specifically handle "Business Rule" violations differently than "Technical Errors."

Q8. (Code-Tracing) Trace the result of this Java-style math logic:

```
try:
    int x = 10 / 0
catch (ArithmeticException e):
    x = 0
print(x)
```

A) 10

B) 0

C) Compilation Error (Variable scope)

D) Error: Division by zero

> In many languages (like Java/C#), variables declared inside a `try` block are local to that block. x cannot be accessed in the `catch` block or the `print` statement unless it was declared *before* the try block.

Q9. (Conceptual) Which of the following is true regarding **Errors** (like `java.lang.Error`)?

A) They should be caught using a `try-catch` block.

B) They are the same as Runtime Exceptions.

C) They represent serious problems that an application should usually not try to catch.

D) They are caused by user input.

> Errors (like `VirtualMachineError`) indicate that the environment is broken. Trying to "catch" and continue from these is usually futile and dangerous.

Q10. (Scenario-Based) A developer needs to ensure a Database connection is closed even if an SQL error occurs. Which block is dedicated to this "cleanup" logic?

A) try

B) catch

C) finally

D) throw

> The `finally` block is the "Cleanup" zone. Whether the operation succeeded or failed, you must release external resources like DB connections, File handles, or Network sockets.

Q11. (Conceptual) Why is it generally considered bad practice to use Exceptions for standard "Control Flow" (e.g., using an exception to break out of a loop)?

A) It is illegal syntax.

B) Generating an exception is computationally expensive because it requires capturing the entire stack trace.

C) Exceptions only work for mathematical errors.

D) It makes the code run too fast for the user.

> Exceptions are for "Exceptional" (unusual) cases. Creating an exception object involves "Stack Walking" to fill the stack trace, which is a very heavy CPU operation. Standard logic should use `if-else` or `break`.

Q12. (Code-Tracing) In a language like Python, what is the equivalent of the `catch` block?

A) handle

B) rescue

C) except

D) error

> Python uses `try...except...finally`. Ruby uses `begin...rescue...ensure`. Though keywords vary, the logic remains identical across OOP languages.

Q13. (Scenario-Based) You are using a `try-with-resources` block (Java) to read a file. When does the `file.close()` method get called?

A) Only if an exception occurs.

B) Only if the file is read successfully.

C) Automatically at the end of the block, regardless of success or failure.

D) Never; you must call it manually.

> `try-with-resources` is "Syntactic Sugar" that ensures any object implementing `AutoCloseable` is closed automatically, preventing resource leaks.

Q14. (Conceptual) What is a "Nested" try-catch block?

A) A try-catch inside a loop.

B) A try-catch block placed inside another try or catch block.

C) A try-catch block that handles two different types of exceptions.

D) A try-catch block written in a child class.

> Nesting is used when a recovery step (inside a `catch` block) might itself cause an error (e.g., trying to log an error to a database, but the database is also down).

Q15. (Debugging) A program crashes with a "Stack Overflow Error." This is most likely caused by:

A) Dividing a number by zero.

B) Trying to access index 10 of a size-5 array.

C) A recursive function that never hits its base case.

D) Creating an object that is too large for the Heap.

> Every function call takes up space on the Stack. Infinite recursion fills this space until the system throws a `StackOverflowError`.

Deep Dive: Under the Hood of Exceptions

1. Stack Unwinding

When an exception is thrown, the runtime begins a process called **Stack Unwinding**. It stops current execution and starts looking at the current stack frame for a `catch` block. If not found, it "pops" the current frame (destroying local variables) and moves to the previous frame (the caller). This continues until a handler is found or the program terminates. This process is why exceptions are "heavy" compared to simple `if` checks.

2. The Cost of `fillInStackTrace()`

In most managed languages (Java/C#), the most expensive part of an exception isn't the `throw` itself, but the creation of the Exception object. The constructor calls a native method (`fillInStackTrace`) that traverses the entire call stack to record exactly where the error happened. In high-frequency trading or high-performance gaming,

developers often "pre-create" exception objects or avoid them entirely to save CPU cycles.

3. Best Practices for Graduates

- **Be Specific:** Never `catch (Exception e)` if you can `catch (IOException e)`. Specificity prevents you from accidentally catching (and hiding) unrelated bugs like `NullPointerExceptions`.
- **Log and Rethrow:** If you can't actually "fix" the problem in the catch block, log the error and rethrow it: `throw e;`. Don't just silence it.
- **Finally for Integrity:** Use `finally` for state cleanup, like setting a `loading = false` flag so the UI doesn't stay frozen after a crash.

Stable Web Resources

- Oracle: Exceptions in Java Official Trail
- Wikipedia: Exception Handling
- Real Python: Mastering Python Exceptions

Quick Revision Cheat Sheet

- **Checked:** Compiler-enforced handling (I/O, SQL).
- **Unchecked:** Runtime logic errors (Null, Math, Array).
- **Finally:** The "Always-Run" block. Ideal for `close()`.
- **Bubbling:** Unhandled exceptions move up the stack to the caller.
- **Anti-pattern:** Swallowing (empty catch) or using exceptions for loops.
- **Stack Trace:** The breadcrumb trail showing where the code failed.
- **Try-with-resources:** Automatic cleanup for modern code.

END OF SUBTOPIC 11 — MOVING TO SUBTOPIC 12

Subtopic 12: Modules, Packages & Libraries (10 MCQs)

Quick Notes

- **Modular Programming:** Dividing a program into separate sub-programs (modules). This promotes *Separation of Concerns* and *Reusability*.
- **Namespaces / Packages:** Containers used to group related classes and functions. They prevent "Naming Collisions" (e.g., having two different classes named `User` in the same project).
- **Static Libraries (.lib, .a):** Code is copied into the executable at compile-time. The resulting file is larger but self-contained.
- **Dynamic Libraries (.dll, .so):** Code is loaded at runtime. Multiple programs can share the same library file, saving disk space and RAM.
- **Dependency Management:** Tools like `npm` (Node), `pip` (Python), `Maven` (Java), and `NuGet` (C#) that automate the downloading and versioning of external libraries.
- **Library vs. Framework:** In a **Library**, *you* call the code (you are in control). In a **Framework**, the *framework* calls your code (Inversion of Control).

Common Traps & Confusions

- **Circular Dependencies:** When Module A imports Module B, and Module B imports Module A. This often causes compilation errors or runtime crashes.
- **Dependency Hell:** A situation where different libraries require conflicting versions of the same third-party package.
- **Global Namespace Pollution:** Declaring too many variables or functions in the "global" scope, making it likely they will accidentally conflict with libraries.

MCQs

Q1. (Conceptual) What is the primary purpose of a **Namespace** in languages like C++, C#, or PHP?

A) To make the code run faster.

B) To prevent naming conflicts by grouping related identifiers under a unique name.

C) To encrypt the source code.

D) To manage the hardware's memory allocation.

> Namespaces allow you to have a `Database.Connect()` and a `Network.Connect()` in the same project. Without namespaces, the compiler wouldn't know which `Connect()` function you are referring to.

Q2. (Comparison) What is a major disadvantage of using a **Static Library** compared to a **Dynamic Library**?

A) It is slower at runtime.

B) It requires a separate file to be present during execution.

C) It increases the size of the executable file and requires a full re-compile to update the library code.

D) It is less secure.

> Because static libraries are physically "baked into" your `.exe`, any update to the library means you must rebuild your entire app. Dynamic libraries (DLLs) can be updated independently without touching the main executable.

Q3. (Scenario-Based) A developer is using `npm` for a JavaScript project and `pip` for a Python project. These tools are best classified as:

A) Compilers

B) Operating Systems

C) Package Managers (Dependency Management)

D) Text Editors

> Package managers automate the process of installing, upgrading, configuring, and removing software libraries (dependencies) that your project needs to function.

Q4. (Conceptual) In software architecture, "Inversion of Control" (IoC) is the technical distinction between:

A) A Class and an Object.

B) A Library and a Framework.

C) A Compiler and an Interpreter.

D) Static and Dynamic typing.

> When you use a Library, you call its functions. When you use a Framework, you plug your code into its "slots," and the framework decides when to call your code. This reversal of the calling flow is called IoC.

Q5. (Debugging) You are trying to compile a project, but the compiler says "Module not found" or "Include file missing," even though the code is perfect. What is the most likely cause?

A) A syntax error in the main function.

B) A logical error in a loop.

C) The "Library Path" or "Search Path" is not correctly configured in the build settings.

D) The computer has run out of RAM.

> Compilers don't search the whole computer for files. You must explicitly tell the compiler which folders contain the external modules or header files your code depends on.

Q6. (Code-Tracing) Look at this Java import: `import java.util.*;`. What does the `*` signify?

A) Import only the classes that contain math.

B) Import every class in the entire Java language.

C) Import all public classes within the `java.util` package.

D) Import only the static methods of the package.

> The wildcard `*` tells the compiler to make every class in that specific package available for use in the current file.

Q7. (Conceptual) What is a **Circular Dependency**?

A) A loop that runs infinitely.

B) When two or more modules depend on each other directly or indirectly, creating a cycle.

C) When an object inherits from its own child.

D) When a function calls itself.

> Circular dependencies make systems "brittle." If Module A needs B to compile, but B needs A to compile, the compiler cannot decide which one to build first, leading to a "chicken and egg" problem.

Q8. (Comparison) In C/C++, what is the role of a **Header File (.h)** compared to a **Source File (.cpp)**?

A) Header files contain the actual logic; Source files contain only comments.

B) Header files contain declarations (signatures); Source files contain definitions (implementations).

C) Header files are only for global variables.

D) Source files are used at runtime; Header files are for compilation.

> Header files act as the "Public Interface." They tell other modules "These functions exist," while the source files contain the secret "How it works" code.

Q9. (Scenario-Based) You want to share your code so that other developers can use it, but you don't want them to see your actual source code. How should you distribute your logic?

A) As a set of `.txt` files.

B) As a pre-compiled Library (e.g., .jar, .dll, or .a file).

C) As a screenshots of your IDE.

D) You cannot share code without sharing the source.

> Libraries allow you to distribute code in its compiled (binary) form. Other developers can link their programs against your library and use your functions without ever seeing your original logic.

Q10. (Conceptual) Which of the following is an example of **Semantic Versioning** (SemVer)?

A) Version "Final_Revised_v2"

B) Version "2023-October"

C) Version "2.4.1" (Major.Minor.Patch)

D) Version "Alpha-Test-01"

> SemVer is the industry standard for libraries. **Major** (breaking changes), **Minor** (new features, no breaks), and **Patch** (bug fixes). This allows developers to safely automate updates.

Deep Dive: The Linking Process and Modular Architecture

1. Static vs. Dynamic Linking (The Binary Level)

When you build a professional application, the **Linker** has two ways to resolve external calls:

- **Static Linking:** The linker extracts the machine code from the `.lib` file and inserts it directly into your `.exe`.
  Pro: The app is "portable" (runs on any machine because it carries its own code).
  Con: Memory waste (if 10 apps use the same library, 10 copies of that code are in RAM).
- **Dynamic Linking:** Your `.exe` only contains a "placeholder" and the name of the `.dll`. When the app starts, the OS finds the DLL and loads it into memory.
  Pro: Shared memory (one copy of the DLL in RAM serves all apps).
  Con: "DLL Hell" (if the DLL is missing or is the wrong version, the app won't start).

2. Modularity vs. Monolith

A **Monolith** is an app where every function is in one big block. A **Modular** app follows the "Plugin" model.
Advantages of Modularity:

1. **Parallel Development:** Team A works on the "Login Module" while Team B works on the "Payment Module."
2. **Maintenance:** You can replace the "Database Module" (e.g., move from MySQL to MongoDB) without touching the "UI Module."
3. **Scalability:** You can reuse the "Email Module" across five different company projects.

3. Dependency Trees

Modern apps often have thousands of dependencies (e.g., Your app → Library A → Library B → Library C). This forms a **Directed Acyclic Graph (DAG)**. If a cycle forms (Circular Dependency), the build system fails. Professional package managers use a **Lockfile** (e.g., `package-lock.json`) to ensure every developer on the team is using the exact same version of every dependency in the tree.

Stable Web Resources

- SemVer.org: Semantic Versioning Specification
- Wikipedia: Modular Programming

Quick Revision Cheat Sheet

- **Module:** An independent part of a program.
- **Namespace/Package:** Prevents naming collisions (Unique folders for code).
- **Static Link:** Code inside the EXE (Self-contained, Large).
- **Dynamic Link:** Code loaded at runtime (Shared, Small, relies on external DLL).
- **Dependency:** An external library your project needs.
- **IoC:** Libraries = You call them. Frameworks = They call you.
- **SemVer (X.Y.Z):** X=Breaking change, Y=Feature, Z=Fix.

END OF SUBTOPIC 12 — MOVING TO SUBTOPIC 13

Subtopic 13: Advanced Programming Concepts (15 MCQs)

Quick Notes

- **Generics / Templates:** Allow a class or function to operate on various data types while providing **compile-time type safety**. (e.g., `List<T>`).
- **Lambdas & Closures:** Anonymous functions that can be treated as data. *Closures* are lambdas that "capture" variables from their surrounding scope.
- **Reflection:** The ability of a program to inspect and modify its own structure (classes, methods, fields) at **runtime**.
- **Annotations / Attributes:** Metadata added to code (e.g., `@Override`, `[Serializable]`) that provides instructions to the compiler or runtime frameworks.

- **Higher-Order Functions:** Functions that take other functions as arguments or return them as results (e.g., `map`, `filter`, `reduce`).
- **Type Erasure:** A Java-specific concept where generic type information is removed after compilation to ensure backward compatibility with older JVM versions.

Common Traps & Confusions

- **Compile-time vs. Runtime:** Generics provide safety at *compile-time*. Reflection allows you to bypass those safety checks at *runtime*, which can lead to "unstable" code.
- **Lambda Scope:** In many languages, a Lambda can read "outer" variables but cannot modify them unless they are effectively final or specifically wrapped (to prevent memory/thread issues).
- **Reflection Performance:** Because Reflection involves looking up strings and metadata at runtime, it is significantly slower than standard direct method calls.

MCQs

Q1. (Conceptual) What is the primary advantage of using **Generics** over using the `Object` class to store items in a collection?

A) It makes the collection take up less RAM.

B) It provides compile-time type safety, eliminating the need for manual type casting and preventing runtime ClassCastExceptions.

C) It allows the collection to store both primitives and objects simultaneously.

D) It automatically encrypts the data inside the collection.

> With `List<String>`, the compiler guarantees only Strings enter the list. Without generics (using `Object`), you could accidentally put an Integer in, and the program would crash only when you try to read it later.

Q2. (Code-Tracing / Java) In Java, what is the result of **Type Erasure** on a `List<Integer>` after the code is compiled into bytecode?

A) It remains `List<Integer>`.

B) It becomes a raw `List` (equivalent to `List<Object>`).

C) It is converted into an `int[]` array.

D) The list is deleted to save space.

> Java uses Type Erasure for backward compatibility. The JVM doesn't actually know about the "Integer" part; the compiler just inserts hidden casts everywhere you access the list.

Q3. (Scenario-Based) You are using a modern Web Framework that automatically creates a database table based on your class name and variable names. Which advanced concept is the framework using to "read" your code?

A) Multi-threading

B) Recursion

C) Reflection

D) Type Casting

> Reflection allows a framework (like Spring, Hibernate, or Entity Framework) to "look" at your class at runtime, see that you named a variable `userName`, and then generate the SQL `CREATE TABLE` command automatically.

Q4. (Code-Tracing) Trace the result of this Higher-Order logic:

```
list = [1, 2, 3, 4]
result = list.filter(x -> x > 2).map(x -> x * 2)
print(result)
```

A) [2, 4, 6, 8]

B) [3, 4]

C) [6, 8]

D) [1, 2, 6, 8]

> 1. `filter(x > 2)` keeps only `[3, 4]`.
> 2. `map(x * 2)` multiplies those values → `[6, 8]`.

Q5. (Conceptual) What is a **Closure** in programming?

A) A function that closes a file handle automatically.

B) A function (usually a lambda) that carries with it the environment (variables) of the scope in which it was created.

C) The final line of a program.

D) A class that cannot be inherited.

> A closure "closes over" the local variables. Even after the outer function finishes, the inner lambda can still access those variables because it has "captured" them.

Q6. (Debugging) Why is **Reflection** often discouraged in performance-critical sections of an application (like the inner loop of a game engine)?

A) It is insecure.

B) It involves significant CPU overhead due to metadata lookups and dynamic type checking that cannot be optimized by the compiler.

C) It cannot handle integers.

D) It requires an internet connection.

> Standard method calls are mapped to direct memory addresses during compilation. Reflection requires the CPU to search through string tables of method names at runtime, which is much slower.

Q7. (Comparison) What is the difference between **C++ Templates** and **Java Generics**?

A) C++ Templates generate a new copy of the code for every type used (Monomorphization); Java uses a single copy of code and erases types.

B) Java Generics are faster.

C) C++ Templates only work with integers.

D) There is no difference.

> C++ creates a `vector_int` and a `vector_double` internally. This makes the binary larger but very fast. Java uses one `List` and treats everything as `Object`, which is slower but keeps the file size smaller.

Q8. (Scenario-Based) A developer adds `@Deprecated` to a method in Java or `[Obsolete]` in C#. What is the purpose of this metadata?

A) To make the method run faster.

B) To warn other developers that the method is outdated and should no longer be used.

C) To hide the method from the public API.

D) To automatically delete the method during the next build.

> These are Annotations/Attributes. They don't change the logic, but they provide info to the compiler to generate warnings and to IDEs to cross out the method name for the user.

Q9. (Code-Tracing) Look at this C++ Template snippet:

```
template <typename T>
T add(T a, T b) { return a + b; }

print(add(5, 10));
print(add(2.5, 1.5));
```

What is the output?

A) 15, 3

B) 15, 4.0

C) Error: Type mismatch

D) 15, 4

> The template generates an `int` version for the first call (15) and a `double` version for the second call (4.0). Templates provide powerful type-specific logic.

Q10. (Conceptual) In functional programming, what does the **Reduce** (or Fold) operation do?

A) It removes elements from a list.

B) It combines all elements of a collection into a single value (e.g., calculating a sum) using a specific accumulator function.

C) It makes the code file smaller.

D) It converts a list into a set.

> Reduce takes a list `[1, 2, 3]` and a function `(a, b) -> a + b` and returns 6. It "reduces" the collection to a single result.

Q11. (Scenario-Based) You want to write a function that can accept any List, but you want to ensure the items in the list are **at least** a type of `Animal` (e.g., `Dog` or `Cat`). Which concept should you use?

A) Reflection

B) Bounded Wildcards / Generic Constraints (e.g., <T extends Animal>)

C) Lambdas

D) Type Erasure

> Bounded generics (Constraints) allow you to restrict the types that can be used with a generic class, ensuring that they share a specific parent or interface.

Q12. (Conceptual) What is **Introspection** in the context of advanced programming?

A) When a developer reviews their own code.

B) The specific part of Reflection that allows a program to examine its own properties (like checking if a class has a certain method).

C) When a program predicts its own crash.

D) A tool for measuring CPU temperature.

> Introspection is the "read-only" part of Reflection. It's used to discover the capabilities of an object at runtime without changing them.

Q13. (Code-Tracing) Trace the value of `result`:

```
def multiplier(factor):
    return lambda x : x * factor

triple = multiplier(3)
result = triple(10)
```

A) 3

B) 10

C) 30

D) Error

> `multiplier(3)` returns a closure where `factor` is 3. When `triple(10)` is called, it executes `10 * 3`, resulting in 30.

Q14. (Conceptual) What is the main purpose of **Fluent Interfaces** (Method Chaining), often seen in advanced libraries?

A) To make the code run faster.

B) To improve code readability by allowing multiple method calls to be linked in a single "sentence-like" instruction.

C) To prevent memory leaks.

D) To use only one variable for the whole program.

> Method chaining (e.g., `object.setName("A").setAge(20).save();`) creates a "Fluent API" that is very popular in modern Java and JavaScript libraries for configuration.

Q15. (Debugging) Why might a **Closure** cause a memory leak in a language with Garbage Collection?

A) Closures cannot be deleted.

B) A long-lived closure might hold a reference to a large object from its outer scope, preventing that object from being garbage collected.

C) Closures use the hard drive instead of RAM.

D) Closures are only stored in the CPU cache.

> Because a closure "captures" variables, those variables stay alive as long as the closure exists. If you store a small lambda in a global list but it captures a 1GB object, that 1GB stays in RAM forever.

Deep Dive: The Engine of Modern Development

1. Generics: Erasure vs. Reification

A graduate must know the "Architectural Battle" between Java and C#/.NET Generics:

- **Java (Type Erasure):** `List<String>` and `List<Integer>` are the SAME class at runtime. You cannot do `instanceof List<String>` because the "String" info is gone.
  Impact: Better backward compatibility, slightly more memory efficient.
- **C#/.NET (Reification):** `List<string>` and `List<int>` are DIFFERENT classes at runtime. You can inspect the type at runtime.
  Impact: Better performance (no casting), allows specialized code for primitives (like `List<int>` being a contiguous block of memory).

2. Reflection: The Framework "Magic"

Frameworks like **Spring (Java)** or **ASP.NET (C#)** use Reflection for **Dependency Injection (DI)**.

1. The framework scans your project for classes with the `@Service` or `[Component]` annotation.
2. Using Reflection, it finds the **Constructor** of those classes.
3. It automatically creates instances and "injects" them where needed.

Without Reflection, developers would have to manually "wire" thousands of objects together using `new` keywords.

3. Functional Patterns in OOP

Modern "Advanced" programming is a hybrid. We use OOP for **Structure** (Classes/Objects) and Functional patterns for **Logic** (Streams/Lambdas).

- **Map:** Transformation (e.g., Convert a list of Users to a list of UserNames).
- **Filter:** Selection (e.g., Keep only users older than 18).
- **Reduce:** Aggregation (e.g., Sum the total price of all items in a cart).

These patterns are **Declarative**: they describe *what* you want to do with the data, rather than *how* to loop through it.

Stable Web Resources

- Oracle: Generics in Java Official Guide
- Wikipedia: Reflection in Computing
- MDN Web Docs: Understanding Closures

Quick Revision Cheat Sheet

- **Generics:** Type safety at compile-time (`List<T>`).
- **Reflection:** Inspecting/Modifying code structure at **runtime**. Great for frameworks, bad for speed.
- **Lambda:** An anonymous function.
- **Closure:** A function + its captured scope.

- **Annotations:** Code metadata (`@Override`).
- **Type Erasure:** Java's way of deleting generic info after build.
- **Higher-Order:** Functions that take/return other functions.
- **Fluent API:** Chainable method calls (`.a().b().c()`).

END OF SUBTOPIC 13 — MOVING TO SUBTOPIC 14

Subtopic 14: Concurrency & Parallelism (10 MCQs)

Quick Notes

- **Concurrency vs. Parallelism:** *Concurrency* is about dealing with many things at once (task switching on one CPU). *Parallelism* is about doing many things at once (running on multiple CPU cores).
- **Process vs. Thread:** A **Process** is an independent program with its own memory space. A **Thread** is a subset of a process; multiple threads share the same memory (Heap) but have their own Stack.
- **Race Condition:** A bug occurring when two threads access shared data simultaneously and the final result depends on the timing of their execution.
- **Deadlock:** A situation where Thread A is waiting for B, and B is waiting for A, causing the program to hang forever.
- **Mutual Exclusion (Mutex):** A mechanism (like a lock) that ensures only one thread can access a critical section of code at a time.
- **Atomic Operations:** Operations that complete in a single step relative to other threads. They cannot be interrupted mid-way.

Common Traps & Confusions

- **The Thread-Safety Myth:** "My code is thread-safe because I use a fast CPU." False. Higher speeds often make race conditions *more* likely and harder to debug.

- **Starvation vs. Deadlock:** In a Deadlock, no one moves. In **Starvation**, the program is running, but one specific thread is perpetually denied the resources it needs to finish.
- **Context Switching Cost:** Creating 10,000 threads for 10,000 tasks is often slower than using 10 threads because the CPU spends all its time swapping threads in and out of memory (Thrashing).

MCQs

Q1. (Conceptual) Which of the following best describes the difference between a Process and a Thread?

A) Processes share the same memory; threads do not.

B) Threads of the same process share the Heap memory but have individual Stacks; Processes have isolated memory.

C) Threads are managed by the hardware; processes are managed by the programmer.

D) A process can only have one thread.

> Threads are "lightweight processes." Because they share the same heap, they can communicate very quickly, but they risk corrupting shared data. Processes are isolated by the OS for security and stability.

Q2. (Scenario-Based) Two threads both execute `count = count + 1` at the exact same time. The initial value is 0. After both finish, the value is 1 instead of 2. This is a classic example of:

A) Deadlock

B) Race Condition

C) Memory Leak

D) Livelock

> Both threads read 0, incremented it to 1, and wrote it back. Because the operation was not **Atomic** or **Synchronized**, one increment was lost.

Q3. (Code-Tracing / Logic) Trace the result of using a **Mutex (Lock)** in this pseudocode:

```
Thread A: Lock(m); x = x + 1; Unlock(m);
Thread B: Lock(m); x = x + 1; Unlock(m);
```

If x starts at 0, what is the guaranteed result?

A) 1

B) 2

C) 0

D) Error

> The Mutex ensures that only one thread can be in the "Critical Section" (x = x + 1) at a time. If Thread A has the lock, Thread B must wait until A unlocks it. This guarantees a correct sum of 2.

Q4. (Conceptual) What are the four necessary conditions for a **Deadlock** to occur (Coffman conditions)?

A) Speed, Memory, CPU, Disk.

B) Loops, If-statements, Recursion, Return.

C) Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

D) Input, Output, Storage, Processing.

> If you break any one of these four conditions (e.g., by allowing resources to be taken away/preempted), a Deadlock becomes mathematically impossible.

Q5. (Comparison) What is the difference between a **Mutex** and a **Semaphore**?

A) Mutex is for integers; Semaphore is for strings.

B) A Mutex is a binary lock (only 1 thread); a Semaphore is a counter that can allow a specific number (N) of threads to access a resource.

C) Semaphores are only used in Python.

D) Mutex is faster than Semaphore.

> Think of a Mutex as a bathroom key (1 person). Think of a Semaphore as a restaurant with 10 tables (10 people can enter; the 11th must wait).

Q6. (Scenario-Based) An application is designed to use 100 threads on a 4-core CPU. The developer notices the app is actually **slower** than the single-threaded version. What is the most likely cause?

A) The CPU is broken.

B) High Context Switching overhead and Thread Contention.

C) 100 is not a multiple of 4.

D) Threads do not work on 4-core CPUs.

> The CPU has to constantly save and load the state of 100 threads. If the tasks are very short, the "management time" (Context Switching) exceeds the "work time," leading to poor performance.

Q7. (Conceptual) What does it mean for a function or class to be **"Thread-Safe"**?

A) It runs only on one thread.

B) It functions correctly during simultaneous execution by multiple threads without data corruption.

C) It cannot be deleted by the OS.

D) It uses the `safe` keyword in C++.

> Thread-safety is achieved through synchronization (locks), using immutable data, or using "Atomic" variables that handle concurrency at the hardware level.

Q8. (Code-Tracing) In Java/C#, what is the effect of calling `thread.join()`?

A) It starts the thread.

B) It makes two threads run in parallel.

C) The calling thread pauses execution until the thread being "joined" has finished its work.

D) It merges two threads into one.

> `Join` is used for synchronization. If the Main thread calls `Worker.join()`, the Main thread will wait at that line until the Worker thread terminates.

Q9. (Conceptual) What is **Amdahl's Law** used to predict?

A) The amount of RAM needed for a thread.

B) The theoretical maximum speedup of a program when using multiple processors, limited by the portion of the program that is strictly sequential.

C) The probability of a deadlock.

D) The lifespan of a CPU.

> If 50% of your code *must* run in a specific order (sequential), then even with an infinite number of processors, you can never speed up the program by more than 2x.

Q10. (Debugging) You have two threads that are constantly changing their state in response to each other, but neither is making any progress toward finishing. This is called:

A) Deadlock
B) Livelock
C) Race Condition
D) Success

> In a Livelock, threads are not "blocked" (waiting), but they are stuck in a loop of reacting to each other (like two people in a hallway trying to step out of each other's way but moving in the same direction repeatedly).

Deep Dive: The Mechanics of Shared State

1. The Critical Section and Mutual Exclusion

A **Critical Section** is any piece of code that accesses a shared resource (like a global variable or a file). To prevent corruption, we enforce **Mutual Exclusion**.
In modern CPUs, this is often implemented using a hardware instruction called **Compare-and-Swap (CAS)**. It checks if a memory location still has the old value before writing the new one. if it changed, the thread "fails" and tries again. This is called **Lock-Free Programming**.

2. Why Deadlocks Happen (The Dining Philosophers)

Imagine 5 philosophers sitting at a round table with 5 forks. To eat, they need 2 forks. If every philosopher picks up the fork on their left at the same time, they all wait for the fork on their right. No one can eat.
Solutions for Graduates:

- **Hierarchy:** Always pick up forks in a specific numerical order.
- **Timeout:** If you can't get the second fork in 5 seconds, put down the first one and try again later.
- **Arbitrator:** Ask a "Waiter" for permission to pick up forks.

3. Thread Pools and Performance

Creating and destroying threads is expensive. Instead of `new Thread().start()`, professional applications use a **Thread Pool**. A fixed number of threads (e.g., matching the number of CPU cores) stay alive and "pick up" tasks from a queue. This eliminates the overhead of thread creation and prevents context switching thrashing.

Stable Web Resources

- Wikipedia: Concurrency in CS
- GeeksforGeeks: Multithreading Explained
- Wikipedia: The Dining Philosophers Problem

Quick Revision Cheat Sheet

- **Thread:** Shares Heap, has own Stack. (Inside a process).
- **Race Condition:** Timing error with shared data. Fix with `Locks`.
- **Deadlock:** Mutual wait (Circular).
- **Atomic:** An operation that cannot be split or interrupted.
- **Mutex:** 1 thread lock. **Semaphore:** N thread limit.
- **Join:** Wait for a thread to finish.
- **Context Switch:** CPU swapping thread states (expensive!).
- **Amdahl's Law:** Sequential code limits the max speedup of parallel code.

END OF SUBTOPIC 14 — MOVING TO SUBTOPIC 15

Subtopic 15: Debugging, Testing & Optimization (15 MCQs)

Quick Notes

- **Levels of Testing:** 1) **Unit Test:** Testing small, individual functions. 2) **Integration Test:** Testing how modules work together. 3) **System Test:** Testing the complete app. 4) **UAT (Acceptance):** Testing by the end-user.
- **Test-Driven Development (TDD):** A process where you write the test *before* the actual code. The cycle is: **Red** (Fail) → **Green** (Pass) → **Refactor**.
- **Black-Box vs. White-Box:** Black-box tests the requirements/UI without seeing code. White-box tests the internal logic, loops, and branches.
- **Regression Testing:** Re-running old tests after a code change to ensure no new bugs were introduced.
- **Profiling:** Using tools to measure which parts of the code use the most CPU or RAM (finding bottlenecks).
- **Linting:** Static analysis tools that flag "Code Smells" (messy code) and potential bugs without actually running the program.

Common Traps & Confusions

- **Testing vs. Debugging:** Testing is the process of *finding* bugs. Debugging is the process of *fixing* them.
- **Premature Optimization:** Optimizing code that isn't slow. As Donald Knuth said, "Premature optimization is the root of all evil." Focus on correctness first.
- **100% Code Coverage Myth:** Having 100% code coverage means every line was executed by a test, but it does NOT mean the logic is bug-free.

MCQs

Q1. (Conceptual) In the "Red-Green-Refactor" cycle of Test-Driven Development (TDD), what does the "Red" stage represent?

A) The program has a syntax error.

B) The code is being deleted.

C) Writing a test for a new feature that fails because the feature hasn't been implemented yet.

D) The server is down.

> In TDD, you write the test first. Since the functionality doesn't exist yet, the test must fail (Red). This ensures the test is actually checking the right thing once the code is written.

Q2. (Scenario-Based) A developer makes a small change to the "Login" logic. Suddenly, the "Checkout" page stops working. Which type of testing would have caught this?

A) Unit Testing

B) Alpha Testing

C) Regression Testing

D) Stress Testing

> Regression testing involves running all existing test cases after a change to ensure that new code hasn't "regressed" or broken old, working functionality.

Q3. (Comparison) What is the primary difference between a **Breakpoint** and a **Watchpoint** during debugging?

A) Breakpoints are for Java; Watchpoints are for C++.

B) A Breakpoint pauses execution at a specific line of code; a Watchpoint pauses execution when a specific variable's value changes.

C) Watchpoints make the code run faster.

D) Breakpoints are only used for syntax errors.

> Use a Breakpoint when you know *where* the error is. Use a Watchpoint when you know *which variable* is being corrupted but don't know which line of code is changing it.

Q4. (Conceptual) In software testing, what is **Boundary Value Analysis**?

A) Testing the program on different operating systems.

B) Testing the speed of the database.

C) Testing inputs at the extreme ends of their allowable range (e.g., testing 0, 1, 99,

and 100 for a range of 1-100).

D) Checking the code for comments.

> Most logical errors (like off-by-one errors) happen at the boundaries of ranges. Testing the minimum, maximum, and values just outside them is a highly effective testing strategy.

Q5. (Scenario-Based) You are optimizing a function that calculates prime numbers. You decide to store previously calculated results in a Hash Map to avoid recalculating them. This optimization is known as:

A) Refactoring

B) Memoization (Caching)

C) Multi-threading

D) Dead-code elimination

> Memoization is a specific form of caching used in dynamic programming. It trades memory (Space) for speed (Time) by storing the results of expensive function calls.

Q6. (Conceptual) What does **Cyclomatic Complexity** measure in a piece of code?

A) The number of lines of code.

B) The memory usage of the function.

C) The number of independent logical paths through the code (IFs, Loops, Cases).

D) How many developers worked on the file.

> Higher cyclomatic complexity means the code has many branches and is harder to test and maintain. A complexity score of over 10 is usually considered a "Code Smell."

Q7. (Debugging) What is a **"Heisenbug"**?

A) A bug that only happens in Python.

B) A bug that seems to disappear or change its behavior when you attempt to study or debug it.

C) A bug caused by a hardware failure.

D) A bug that is found by an end-user.

> These often occur in multi-threaded code or timing-sensitive logic. The act of adding a `print` statement or using a debugger changes the timing of the system, making the bug vanish.

Q8. (Scenario-Based) An application works perfectly for 1 user, but the response time becomes 30 seconds when 1,000 users log in. Which tool should the developer use to find the cause?

A) A Linter

B) A Compiler

C) A Profiler

D) A Text Editor

> A Profiler tracks execution time and memory usage per function. It would reveal which specific part of the code (e.g., a slow DB query) is causing the bottleneck under high load.

Q9. (Conceptual) What is the purpose of an **Assertion** (e.g., `assert x > 0`) in programming?

A) To print data to the user.

B) To state a condition that must be true at a specific point in the code; if false, the program terminates immediately to signal a bug.

C) To handle user input errors.

D) To speed up the compilation.

> Assertions are "Internal checks" for developers. They are used to catch "impossible" states during development. They are often disabled in the final production release.

Q10. (Comparison) What is the difference between **Alpha Testing** and **Beta Testing**?

A) Alpha is done by internal employees; Beta is done by a group of real external users.

B) Alpha is for mobile; Beta is for web.

C) Beta testing is done before Alpha testing.

D) Alpha testing checks for security; Beta checks for speed.

> Alpha testing is the final internal check. Beta testing is a "Field Test" where the software is released to a limited audience of real users to find bugs in real-world environments.

Q11. (Conceptual) Which optimization technique involves replacing a function call with the actual body of the function during compilation?

A) Loop Unrolling

B) Function Inlining

C) Garbage Collection

D) Recursion

> Inlining eliminates the overhead of jumping to a new memory address and setting up a stack frame. It is great for tiny, frequently called functions but increases binary size.

Q12. (Scenario-Based) A developer uses a "Mock Object" during a Unit Test. Why?

A) To make the test run on a different server.

B) To simulate a complex dependency (like a Database or API) so the unit can be tested in isolation.

C) To encrypt the test data.

D) Because the real object is too small.

> Unit tests should be independent. If you want to test "calculateTotal()", you shouldn't need a real database. You use a "Mock" that just returns a fixed number when asked for a price.

Q13. (Conceptual) What is **Fuzz Testing** (Fuzzing)?

A) Testing the program with very simple inputs.

B) Providing massive amounts of random, invalid, or unexpected data to a program to find security vulnerabilities or crashes.

C) Testing the user interface colors.

D) Measuring the sound the CPU makes.

> Fuzzing is an automated technique used to find buffer overflows and logic gaps by bombarding an input field with millions of "garbage" combinations.

Q14. (Debugging) You see a **"Core Dump"** file after a C++ program crashes. What does this file contain?

A) A copy of the source code.

B) A snapshot of the program's memory (RAM) at the exact moment it crashed.

C) A list of all user passwords.

D) The instructions to restart the computer.

> A Core Dump allows a developer to "post-mortem" debug. You can load the file into a debugger to see exactly what the values of all variables were when the crash happened.

Q15. (Scenario-Based) A linter flags a "Dead Code" warning. What does this mean?

A) The code contains a virus.

B) The code is written in an old language.

C) The code can never be reached or executed during the program's run (e.g., code after a `return`).

D) The computer's hardware is failing.

> Dead code (unreachable code) litters the codebase, increases file size, and confuses other developers. It should be removed.

Deep Dive: The Science of Quality Assurance

1. The Testing Pyramid

A graduate engineer should design a testing strategy based on the **Testing Pyramid**:

- **Base (Unit Tests):** Thousands of fast, cheap tests for individual logic. Should make up 70% of your suite.
- **Middle (Service/Integration Tests):** Tests how your API talks to the Database or other services.
- **Top (UI/E2E Tests):** Slow, expensive tests that click through the actual browser. Keep these to a minimum (only the "Happy Path").

2. Cyclomatic Complexity (McCabe's Metric)

This is a quantitative measure of the number of linearly independent paths through a program's source code.
Formula: `M = E - N + 2P`
(E = edges, N = nodes, P = connected components).
In simple terms: Every `if`, `while`, `for`, and `case` adds 1 to the complexity. A high score means a function is doing too much and should be broken down (Decomposed).

3. Profiling: Sampling vs. Instrumentation

How do profilers measure code?

- **Sampling:** The profiler "looks" at the CPU every 1ms to see which function is running. Low overhead, but less accurate for very fast functions.
- **Instrumentation:** The profiler adds "counters" into your code (e.g., `start_timer()` and `end_timer()`). Very accurate, but slows down the program significantly during the test.

Stable Web Resources

- Wikipedia: Software Testing
- Martin Fowler: The Practical Test Pyramid
- Guru99: Cyclomatic Complexity Explained

Quick Revision Cheat Sheet

- **TDD:** Red → Green → Refactor.
- **Unit:** Smallest part. **Integration:** Interaction. **Regression:** Don't break old stuff.
- **Black-Box:** No code access. **White-Box:** Full code access.
- **Breakpoint:** Stops at Line. **Watchpoint:** Stops at Variable Change.
- **Linter:** Static check (looks for "smells"). **Profiler:** Dynamic check (looks for "slowness").
- **Memoization:** Cache the result of expensive functions.
- **Assertion:** `assert x != null`; Catch developer mistakes early.

END OF SUBTOPIC 15 — MOVING TO SUBTOPIC 16

Subtopic 16: Software Development Practices (10 MCQs)

Quick Notes

- **Version Control (Git):** A system that records changes to a file or set of files over time so that you can recall specific versions later. *Distributed* means every developer has a full copy of the history.
- **CI/CD (Continuous Integration / Continuous Deployment):** An automated pipeline where code is automatically built, tested, and deployed to production whenever a change is made.
- **Agile / Scrum:** An iterative approach to software development. Work is divided into 2-4 week **Sprints**. Key events include Daily Standups, Sprint Planning, and Retrospectives.
- **Code Review:** A process where developers check each other's code for bugs, style issues, and logic errors before it is merged into the main codebase.
- **Technical Debt:** The long-term cost of choosing an "easy" solution now instead of a "better" one that would take longer to implement.
- **Pair Programming:** Two developers working at one workstation. One writes code (The Driver) while the other reviews each line as it is typed (The Navigator).

Common Traps & Confusions

- **Commit vs. Push:** A `commit` saves changes locally on your machine. A `push` sends those saved changes to a remote server (like GitHub) for others to see.
- **Merge vs. Rebase:** `Merge` joins two histories together with a single "Merge Commit." `Rebase` rewrites history by moving your changes to the top of the latest code (cleaner but more dangerous for shared branches).
- **Git Reset vs. Git Revert:** `Reset` "deletes" commits from history (unsafe for public code). `Revert` creates a *new* commit that does the exact opposite of a previous commit (safe).

MCQs

Q1. (Conceptual) In Git, what is the state of a newly created file that has not yet been added to the staging area?

A) Staged
B) Modified
C) Untracked
D) Committed

> A new file is "Untracked" by Git until you run `git add`. Once added, it becomes "Staged." After `git commit`, it is stored in the local repository.

Q2. (Scenario-Based) Two developers edit the exact same line of the same file and try to merge their changes. Git stops the process and asks for help. This situation is called a:

A) Syntax Error
B) Runtime Exception
C) Merge Conflict
D) Pull Request Denial

> Git is smart, but it cannot decide which version of a specific line is "correct." A human must manually resolve the conflict by picking the right code and committing the result.

Q3. (Conceptual) What is the primary goal of **Continuous Integration (CI)**?

A) To deploy code to the user as fast as possible.
B) To detect bugs early by automatically building and testing the code every time a developer commits a change.
C) To prevent developers from working on the same file.
D) To encrypt the source code repository.

> CI ensures that the "Main" branch is always healthy. If a developer pushes code that breaks a test, the CI server flags it immediately before the bug can reach production.

Q4. (Scenario-Based) In a **Scrum** team, the developers meet for 15 minutes every morning to discuss: 1) What they did yesterday, 2) What they will do today, and 3) Any blockers. This is called:

A) Sprint Planning

B) Retrospective

C) Daily Standup

D) Code Review

> The Daily Standup is a "sync" meeting designed to keep the team aligned and identify obstacles quickly.

Q5. (Comparison) What is a major advantage of **Distributed** Version Control (like Git) over **Centralized** Version Control (like SVN)?

A) It uses less disk space.

B) Developers can work, commit, and view history offline because they have a full copy of the repository on their own machine.

C) It is only compatible with Linux.

D) It prevents merge conflicts from happening.

> In a distributed system, you don't need a constant internet connection to the server to check the history or save your progress. You only need the server when you want to `push` or `pull`.

Q6. (Conceptual) What is a **"Pull Request"** (or Merge Request)?

A) A request for a salary increase.

B) A command to download code from a server.

C) A formal request to merge changes from one branch into another, usually involving a code review process.

D) A request to restart the server.

> Pull Requests are the heart of professional collaboration. They allow other team members to review your code, suggest changes, and ensure quality before the code is officially merged.

Q7. (Scenario-Based) You accidentally committed a sensitive API key to a public GitHub repository. What is the safest way to fix this?

A) Delete the file and commit again.

B) Change the file and commit again.

C) Invalidate/Rotate the API key immediately and then use a tool to scrub the

history of the repository.

D) Do nothing; Git is secure.

> Simply deleting the key in a *new* commit is not enough because the key remains visible in the *previous* commits in the history. You must rotate the key (make it useless) and then clean the history.

Q8. (Conceptual) In software maintenance, what is **"Perfective Maintenance"**?

A) Fixing a crash reported by a user.

B) Improving the performance or maintainability of the software without changing its functionality (e.g., refactoring).

C) Updating the software to work on a new OS.

D) Adding a new feature requested by a client.

> Perfective maintenance is about making the software "better" internally—optimizing speed, cleaning code, or improving the UI—to provide a better experience.

Q9. (Comparison) What is the difference between **Git Revert** and **Git Reset --hard**?

A) There is no difference.

B) Revert creates a new "undo" commit; Reset deletes commits and changes the actual history.

C) Reset is safer for shared branches.

D) Revert is only for single files.

> `Revert` is the "Professional" way to undo. It keeps the history intact. `Reset` is the "Destructive" way; if you reset a branch that others have already downloaded, you will break their local versions.

Q10. (Conceptual) What does the term **"Definition of Done" (DoD)** mean in professional software development?

A) The developer finished typing the code.

B) The code was pushed to GitHub.

C) A checklist of requirements (e.g., code reviewed, tests passed, documented) that must be met before a task is considered finished.

D) The project is sold to a customer.

> DoD prevents "it works on my machine" syndrome. It ensures that quality standards are met for every single feature before it is officially called "done."

Deep Dive: Professional Workflow

1. Git-Flow: Branching Strategy

Graduate engineers don't just "push to main." They use a strategy like **Git-Flow**:

- **Main/Master:** Always contains stable, production-ready code.
- **Develop:** The "Integration" branch where all features are merged for testing.
- **Feature Branches:** Temporary branches where individual tasks are coded (e.g., `feature/login-ui`).
- **Hotfix Branches:** Emergency branches created from Main to fix a critical production bug immediately.

2. The CI/CD Pipeline Architecture

A modern pipeline follows these automated steps:

1. **Source:** Triggered when code is pushed to Git.
2. **Build:** Compiles the code and checks for syntax errors.
3. **Test:** Runs Unit Tests and Integration Tests. If any fail, the pipeline stops.
4. **Security Scan:** Checks libraries for known vulnerabilities (CVEs).
5. **Deploy:** Automatically pushes the binary to a Staging or Production server.

3. Software Maintenance Categories

Type	Purpose	Example
Corrective	Fixing existing bugs.	Fixing a NullPointerException.
Adaptive	Changing to fit a new environment.	Updating code to support Windows 11.
Perfective	Enhancing performance/ readability.	Refactoring a slow loop.

Preventive	Preventing future issues.	Updating an old library before it becomes unsupported.

Stable Web Resources

- Pro Git Book: The Definitive Guide to Git
- Atlassian: Kanban vs. Scrum Explained

Quick Revision Cheat Sheet

- **Untracked:** New file Git doesn't know about.
- **Staging Area:** The "Waiting Room" for files before committing.
- **Commit:** Local snapshot. **Push:** Sending to server.
- **Sprint:** A fixed time block in Agile (2-4 weeks).
- **Standup:** 15-min daily sync meeting.
- **Code Review:** Peer check to ensure quality.
- **CI:** Auto-test every commit. **CD:** Auto-deploy to users.
- **Merge Conflict:** Two people edited the same line.
- **Technical Debt:** Quick fixes that cost time later.